

Fundamentos y Aplicaciones de la Criptografía Moderna: Desde la Representación de la Información hasta TLS 1.3

Resumen

La criptografía constituye uno de los pilares fundamentales de la seguridad informática moderna, ya que permite proteger la información frente a accesos no autorizados, modificaciones maliciosas y suplantación de identidad. En la actualidad, prácticamente todos los servicios digitales dependen de mecanismos criptográficos para garantizar la confidencialidad, la integridad, la autenticidad y, en determinados contextos, el no repudio de la información. Desde aplicaciones de mensajería instantánea hasta sistemas bancarios, comercio electrónico y protocolos de comunicación seguros, la criptografía proporciona las herramientas necesarias para establecer confianza en entornos abiertos como Internet.

El presente trabajo desarrolla de manera progresiva los fundamentos matemáticos y criptográficos que sustentan los sistemas modernos de protección de la información. Se inicia con el estudio de la representación digital de los datos y de los conceptos matemáticos esenciales, tales como la aritmética modular y los inversos modulares, para posteriormente abordar los sistemas clásicos de cifrado basados en sustitución y transposición. Posteriormente, se estudian los algoritmos de criptografía simétrica modernos, haciendo especial énfasis en

AES, sus fundamentos matemáticos, sus modos de operación y los principios de un diseño criptográfico seguro.

Asimismo, se analizan las funciones hash, los códigos de autenticación de mensajes (MAC y HMAC), los mecanismos de derivación de llaves, los sistemas criptográficos asimétricos como Diffie-Hellman, RSA y la criptografía basada en curvas elípticas, así como las firmas digitales, los certificados digitales y el protocolo TLS 1.3, considerado actualmente el estándar para la protección de las comunicaciones en Internet.

El propósito de este documento es presentar una visión integral del funcionamiento de la criptografía moderna, destacando la relación existente entre los fundamentos matemáticos, los algoritmos criptográficos y sus aplicaciones prácticas en sistemas reales.

Palabras clave: criptografía, AES, RSA, Diffie-Hellman, TLS, funciones hash, firmas digitales, certificados digitales, curvas elípticas.

Introducción

La creciente dependencia de los sistemas digitales ha convertido la protección de la información en una necesidad indispensable para personas, empresas y gobiernos. Cada día se transmiten millones de datos sensibles a través de redes públicas, incluyendo información financiera, documentos personales, credenciales de acceso y comunicaciones privadas. Sin mecanismos adecuados de protección, estos datos podrían ser interceptados, modificados o utilizados por terceros con fines maliciosos.

La criptografía surge precisamente como la disciplina encargada de desarrollar técnicas matemáticas capaces de proteger la información frente a diversos tipos de amenazas. Aunque históricamente fue utilizada principalmente con fines militares y diplomáticos, actualmente constituye una herramienta

indispensable para prácticamente cualquier sistema informático conectado a Internet.

Los sistemas criptográficos modernos ya no se limitan únicamente al cifrado de mensajes. También permiten verificar la integridad de los datos, autenticar usuarios y servidores, generar llaves seguras, firmar documentos electrónicos y establecer canales de comunicación protegidos entre dispositivos que nunca antes han compartido información secreta.

Para comprender estos mecanismos resulta necesario estudiar primero los principios matemáticos sobre los cuales fueron contruidos. Conceptos aparentemente sencillos como los residuos de una división, la representación binaria de la información o determinadas propiedades algebraicas constituyen la base sobre la cual funcionan algoritmos ampliamente utilizados en la actualidad, como AES, RSA o Diffie-Hellman.

En este trabajo se presenta un recorrido progresivo desde los fundamentos matemáticos hasta los protocolos de comunicación seguros empleados actualmente en Internet, mostrando cómo cada uno de estos elementos se integra dentro de un sistema criptográfico completo.

Capítulo 1. Introducción a la criptografía

1.1 Concepto de criptografía

La criptografía es la disciplina encargada de proteger la información mediante técnicas matemáticas que permiten transformar los datos originales en representaciones que únicamente puedan ser comprendidas por los usuarios autorizados. Su objetivo principal consiste en garantizar que la información conserve determinadas propiedades de seguridad durante su almacenamiento o transmisión.

Aunque frecuentemente se asocia únicamente con el cifrado de mensajes, la criptografía moderna abarca un conjunto mucho más

amplio de mecanismos orientados a resolver distintos problemas relacionados con la seguridad de la información.

Las principales propiedades que busca proteger son:

- Confidencialidad.
- Integridad.
- Autenticidad.
- No repudio (en determinados contextos).

La confidencialidad asegura que únicamente las personas autorizadas puedan conocer el contenido de un mensaje. Para lograrlo se utilizan algoritmos de cifrado que transforman la información original en un formato ilegible para cualquier tercero que no posea la llave correspondiente.

La integridad permite detectar cualquier modificación realizada sobre los datos después de haber sido generados o transmitidos. Esta propiedad resulta esencial para impedir que un atacante altere información sin que el receptor pueda detectarlo.

La autenticidad garantiza que la información realmente proviene de quien afirma haberla enviado. En los sistemas modernos esta propiedad suele obtenerse mediante mecanismos como firmas digitales, certificados electrónicos y protocolos de autenticación.

Finalmente, el no repudio impide que el autor de determinada información pueda negar posteriormente haberla generado, siempre que se utilicen mecanismos criptográficos adecuados.

Es importante destacar que estas propiedades no siempre son proporcionadas por un único algoritmo. En la práctica, los sistemas modernos combinan diversas primitivas criptográficas para ofrecer protección integral frente a múltiples amenazas.

1.2 Evolución de la criptografía

El desarrollo histórico de la criptografía puede dividirse en dos grandes etapas: la criptografía clásica y la criptografía

moderna.

La criptografía clásica trabajaba principalmente con letras y símbolos del alfabeto. Los mensajes eran transformados mediante reglas relativamente simples, como desplazar letras, sustituir caracteres o reorganizar su posición dentro del texto. Este tipo de algoritmos era adecuado para los medios de comunicación disponibles en su época, pero presentaba vulnerabilidades importantes frente al análisis estadístico y otras técnicas de criptoanálisis.

Con el desarrollo de las computadoras surgió la necesidad de diseñar algoritmos mucho más robustos. La criptografía moderna dejó de trabajar con letras y comenzó a operar directamente sobre bits y bytes. Este cambio permitió aprovechar propiedades matemáticas mucho más complejas y construir algoritmos capaces de resistir ataques computacionales extremadamente sofisticados.

Actualmente, los sistemas criptográficos modernos combinan algoritmos simétricos, algoritmos asimétricos, funciones hash, protocolos de intercambio de llaves y mecanismos de autenticación para proporcionar seguridad en aplicaciones utilizadas diariamente por millones de personas.

1.3 Objetivos de la criptografía moderna

La criptografía moderna no pretende únicamente ocultar información. Su propósito consiste en resolver diferentes problemas relacionados con la seguridad digital.

Entre sus principales objetivos se encuentran:

- Proteger la confidencialidad de la información.
- Detectar modificaciones no autorizadas.
- Verificar la identidad de usuarios y servidores.
- Permitir el intercambio seguro de llaves.
- Proteger comunicaciones en redes públicas.
- Garantizar la autenticidad de documentos electrónicos.

- Establecer canales seguros para aplicaciones distribuidas.

Cada uno de estos objetivos requiere mecanismos criptográficos específicos. Por ejemplo, mientras que el cifrado simétrico resulta altamente eficiente para proteger grandes cantidades de información, los algoritmos asimétricos facilitan el intercambio seguro de llaves y la autenticación de usuarios. De manera similar, las funciones hash permiten verificar la integridad de archivos, mientras que las firmas digitales proporcionan autenticidad y, en determinados contextos, no repudio.

En consecuencia, la seguridad de un sistema moderno no depende de un único algoritmo, sino de la correcta combinación de múltiples herramientas criptográficas que trabajan conjuntamente para satisfacer los diferentes requisitos de seguridad.

Capítulo 2. Fundamentos Matemáticos de la Criptografía

2.1 La información digital como base de la criptografía

Antes de estudiar los algoritmos criptográficos modernos es indispensable comprender cómo representan la información las computadoras. A diferencia de las personas, que interpretan letras, números, imágenes o sonidos como elementos distintos, un sistema digital únicamente procesa secuencias de bits. En consecuencia, toda la información utilizada por una computadora termina siendo representada mediante combinaciones de ceros y unos.

El bit constituye la unidad mínima de información digital y puede adoptar únicamente dos estados posibles: 0 o 1. A partir de la combinación de varios bits se construyen unidades de mayor tamaño denominadas bytes, donde un byte corresponde a ocho bits. Esta representación permite almacenar caracteres, números, imágenes, archivos de audio, videos y cualquier otro tipo de información.

Por ejemplo, una palabra aparentemente simple como "HOLA" no es interpretada directamente como texto por un procesador. Cada carácter es convertido primero en un valor numérico mediante una codificación estándar, posteriormente representado en formato binario y finalmente procesado como una secuencia de bits. Este principio constituye el fundamento sobre el cual trabajan todos los algoritmos criptográficos modernos.

La criptografía actual no cifra palabras ni frases en el sentido tradicional, sino bloques de información binaria. Este cambio representa una diferencia fundamental respecto a la criptografía clásica, donde los algoritmos operaban directamente sobre letras del alfabeto. Al trabajar con bits y bytes, los sistemas modernos pueden proteger cualquier tipo de información independientemente de su naturaleza.

La representación binaria también permite expresar la misma información utilizando otros sistemas numéricos ampliamente empleados en informática, como la notación hexadecimal y la codificación Base64. Estas representaciones no modifican el contenido original de los datos; simplemente ofrecen formas distintas de visualizar o transportar la misma información binaria. El uso de hexadecimal resulta especialmente frecuente en criptografía debido a que facilita la lectura de secuencias largas de bytes, mientras que Base64 suele utilizarse para transmitir datos binarios mediante protocolos diseñados originalmente para texto.

Comprender esta forma de representación resulta esencial, ya que algoritmos como AES, SHA-256 o RSA operan directamente sobre secuencias binarias y no sobre caracteres individuales.

2.2 Sistemas de numeración utilizados en criptografía

La informática utiliza diferentes sistemas de numeración según las necesidades del procesamiento de datos. Aunque el sistema decimal es el más familiar para las personas, los sistemas

criptográficos emplean principalmente las bases binaria y hexadecimal debido a su estrecha relación con la arquitectura de las computadoras.

El sistema binario utiliza únicamente los dígitos 0 y 1. Cada posición representa una potencia de dos, permitiendo expresar cualquier número mediante combinaciones binarias. Este sistema constituye el lenguaje natural del hardware, ya que los circuitos electrónicos únicamente distinguen entre dos estados eléctricos.

El sistema hexadecimal utiliza dieciséis símbolos: los números del 0 al 9 y las letras A hasta F. Cada dígito hexadecimal representa exactamente cuatro bits, lo que simplifica considerablemente la representación de grandes cantidades de información binaria. Por esta razón, numerosas llaves criptográficas, hashes y textos cifrados suelen mostrarse en formato hexadecimal.

Por otra parte, Base64 constituye un mecanismo de codificación diseñado para representar datos binarios utilizando caracteres imprimibles. Su finalidad no es proporcionar seguridad ni cifrado, sino facilitar la transmisión de información binaria mediante protocolos que originalmente sólo aceptaban texto.

Es importante distinguir claramente entre codificación y cifrado. Mientras que una codificación únicamente cambia la forma en que se representa la información, un algoritmo criptográfico transforma los datos con el propósito de impedir su comprensión por personas no autorizadas.

2.3 Aritmética modular

Uno de los conceptos matemáticos más importantes en criptografía es la aritmética modular. A diferencia de la aritmética convencional, donde los números pueden crecer indefinidamente, la aritmética modular trabaja dentro de un conjunto finito de valores. Cuando un resultado alcanza el valor del módulo, vuelve a comenzar desde cero.

Este comportamiento puede compararse con el funcionamiento de un reloj. Después de las doce horas vuelve nuevamente la una, iniciando un nuevo ciclo. De manera similar, al trabajar módulo cinco únicamente existen cinco residuos posibles: cero, uno, dos, tres y cuatro. Cualquier operación realizada dentro de este sistema terminará produciendo alguno de estos valores.

Formalmente, la operación módulo obtiene el residuo de la división entera entre dos números. Si un número se divide entre otro, el residuo constituye el resultado de la operación modular. Esta característica permite construir sistemas matemáticos cerrados que presentan propiedades extremadamente útiles para el diseño de algoritmos criptográficos.

Por ejemplo, al dividir diecisiete entre cinco se obtiene un cociente de tres y un residuo de dos. En consecuencia, se dice que diecisiete módulo cinco es igual a dos. Del mismo modo, veintidós y veintisiete producen exactamente el mismo residuo al dividirse entre cinco, razón por la cual ambos números pertenecen a la misma clase de congruencia.

La aritmética modular constituye el fundamento matemático de numerosos algoritmos modernos. RSA basa gran parte de su funcionamiento en operaciones modulares con números extremadamente grandes, mientras que Diffie-Hellman utiliza exponenciación modular para establecer secretos compartidos. Incluso los sistemas de curvas elípticas emplean operaciones modulares sobre cuerpos finitos para construir algoritmos más eficientes.

2.4 Congruencia modular

La congruencia modular permite determinar cuándo dos números se comportan de la misma manera respecto a un determinado módulo. Dos valores son congruentes si producen exactamente el mismo residuo después de dividirse entre el módulo correspondiente.

Esta propiedad permite reemplazar números muy grandes por otros mucho más pequeños sin modificar el resultado final de

determinadas operaciones. Gracias a ello, los algoritmos criptográficos pueden realizar cálculos extremadamente complejos manteniendo controlado el tamaño de los valores intermedios.

Las operaciones de suma, resta y multiplicación conservan esta propiedad de congruencia, lo que hace posible construir algoritmos eficientes para el manejo de números de cientos o incluso miles de bits. Sin estas propiedades matemáticas sería prácticamente imposible implementar sistemas como RSA o Diffie-Hellman de forma eficiente.

La utilización de congruencias también permite simplificar numerosos cálculos durante el análisis criptográfico y constituye uno de los pilares fundamentales del álgebra utilizada en seguridad informática.

2.5 Inverso modular

En la aritmética convencional, el inverso multiplicativo de un número es aquel que al multiplicarse por el valor original produce como resultado la unidad. Sin embargo, dentro de la aritmética modular este concepto requiere una adaptación.

El inverso modular de un número corresponde al valor que, multiplicado por dicho número y reducido módulo un determinado entero, produce como resultado uno. No todos los números poseen un inverso modular; su existencia depende de ciertas propiedades matemáticas relacionadas con el máximo común divisor entre el número y el módulo utilizado.

La importancia del inverso modular radica en que permite "deshacer" determinadas operaciones de multiplicación dentro del sistema modular. Esta capacidad resulta indispensable para construir algoritmos criptográficos de llave pública.

Por ejemplo, RSA requiere calcular inversos modulares durante el proceso de generación de llaves. Sin esta operación sería imposible obtener la llave privada correspondiente a una llave pública determinada. En consecuencia, aunque el concepto pueda parecer puramente matemático, constituye una pieza esencial del

funcionamiento interno de uno de los sistemas criptográficos más utilizados en el mundo.

2.6 Teoremas fundamentales

La criptografía moderna descansa sobre diversos resultados matemáticos que permiten demostrar el correcto funcionamiento de numerosos algoritmos. Entre ellos destacan los relacionados con números primos, divisibilidad, máximo común divisor y propiedades de las congruencias.

Estos teoremas establecen las condiciones bajo las cuales determinadas operaciones poseen solución y permiten garantizar que los algoritmos funcionen de manera consistente para cualquier conjunto válido de parámetros. En particular, los sistemas criptográficos de llave pública dependen directamente de propiedades algebraicas que únicamente pueden demostrarse mediante estos resultados matemáticos.

Lejos de constituir simples herramientas teóricas, estos principios proporcionan la base formal sobre la cual se diseñan algoritmos utilizados diariamente para proteger millones de comunicaciones en Internet. Sin ellos no sería posible construir mecanismos confiables para el intercambio de llaves, la generación de firmas digitales o el establecimiento de protocolos seguros.

La comprensión de estos fundamentos matemáticos marca la transición entre la criptografía clásica y la criptografía moderna. Mientras los sistemas históricos dependían principalmente del secreto de sus métodos, los algoritmos actuales basan su seguridad en problemas matemáticos cuya resolución resulta computacionalmente inviable con la tecnología disponible.

Capítulo 3. Criptografía Clásica

3.1 Introducción

La criptografía clásica representa el origen de las técnicas utilizadas para proteger la información antes del desarrollo de las computadoras. Durante siglos, gobiernos, ejércitos y organizaciones recurrieron a diversos métodos para impedir que mensajes importantes fueran comprendidos por personas no autorizadas. Aunque estos sistemas resultan sencillos comparados con los algoritmos actuales, introdujeron conceptos fundamentales que siguen presentes en la criptografía moderna, como el uso de llaves, la transformación de la información y la necesidad de mantener ciertos elementos en secreto.

Los primeros sistemas criptográficos fueron diseñados para trabajar directamente sobre caracteres del alfabeto. En ellos, cada letra podía ser sustituida, desplazada o reorganizada siguiendo un conjunto específico de reglas. El objetivo era producir un mensaje aparentemente incomprensible para cualquier persona que desconociera el procedimiento utilizado para generar el texto cifrado.

Con el paso del tiempo surgieron técnicas de análisis capaces de romper muchos de estos sistemas, especialmente mediante el estudio de la frecuencia con la que aparecen determinadas letras dentro de un idioma. Estas limitaciones impulsaron el desarrollo de mecanismos cada vez más complejos, culminando finalmente en la criptografía basada en fundamentos matemáticos utilizada en la actualidad.

3.2 Conceptos fundamentales

Todo sistema criptográfico clásico puede describirse mediante cuatro elementos principales: el texto plano, el algoritmo de cifrado, la llave y el texto cifrado.

El **texto plano** corresponde a la información original que se desea proteger. Puede tratarse de una carta, una orden militar, un documento o cualquier mensaje que deba permanecer confidencial.

El **algoritmo de cifrado** es el conjunto de reglas que transforma el texto original en una representación diferente. Estas reglas

especifican cómo deben modificarse las letras para producir un mensaje ilegible.

La **llave** constituye el parámetro que determina el comportamiento específico del algoritmo. Aunque dos personas utilicen el mismo método de cifrado, obtendrán resultados distintos si emplean llaves diferentes. Este concepto continúa siendo esencial en la criptografía moderna.

Finalmente, el **texto cifrado** es el resultado obtenido después de aplicar el algoritmo al mensaje original. Su contenido carece de significado para cualquier persona que desconozca la llave utilizada durante el proceso de cifrado.

El receptor autorizado puede recuperar el mensaje original aplicando el proceso inverso, conocido como descifrado, utilizando la misma llave o una llave relacionada, dependiendo del sistema empleado.

3.3 Cifrados por sustitución

Los cifrados por sustitución constituyen una de las técnicas criptográficas más antiguas. Su funcionamiento consiste en reemplazar cada símbolo del mensaje por otro símbolo siguiendo una regla previamente establecida.

En este tipo de algoritmos la posición de las letras permanece inalterada; únicamente cambia el símbolo que ocupa cada lugar. La seguridad depende de la dificultad para descubrir la correspondencia utilizada entre el alfabeto original y el alfabeto cifrado.

Uno de los ejemplos más conocidos es el **cifrado César**, atribuido al emperador romano Julio César. En este método cada letra del mensaje es desplazada un número fijo de posiciones dentro del alfabeto. Si el desplazamiento corresponde a tres posiciones, la letra A se convierte en D, la B en E y así sucesivamente hasta completar el alfabeto.

Aunque este procedimiento representa un avance importante respecto a la escritura en texto claro, posee una debilidad evidente: únicamente existen un número reducido de posibles desplazamientos. Un atacante puede probar todas las combinaciones posibles en poco tiempo, recuperando el mensaje original sin necesidad de conocer la llave.

Posteriormente aparecieron sistemas de sustitución mucho más complejos, donde cada letra podía asociarse con cualquier otra del alfabeto. Estos métodos incrementaban considerablemente el número de posibles llaves, haciendo más difícil el ataque por fuerza bruta.

Sin embargo, incluso estos sistemas presentan vulnerabilidades importantes frente al análisis estadístico. En cualquier idioma existen letras que aparecen con mayor frecuencia que otras. Por ejemplo, en español las vocales y determinadas consonantes son considerablemente más comunes que otras letras. Si un atacante analiza suficientes mensajes cifrados, puede identificar estos patrones y reconstruir gradualmente el alfabeto utilizado.

Esta limitación demostró que aumentar únicamente el número de sustituciones posibles no era suficiente para garantizar un nivel adecuado de seguridad.

3.4 Cifrados por transposición

A diferencia de los algoritmos de sustitución, los cifrados por transposición conservan todas las letras originales del mensaje, modificando únicamente su posición.

El procedimiento consiste en reorganizar los caracteres siguiendo una regla determinada por la llave utilizada. Como resultado, el mensaje conserva exactamente los mismos símbolos, aunque distribuidos en un orden completamente distinto.

Este tipo de algoritmos evita parcialmente algunos problemas derivados del análisis de frecuencia, ya que las letras no son sustituidas por otras. No obstante, la frecuencia de aparición de cada símbolo permanece exactamente igual, permitiendo que un

atacante experimentado pueda reconstruir progresivamente la estructura original del mensaje.

Diversos sistemas históricos combinaron sustitución y transposición con el propósito de incrementar la dificultad del criptoanálisis. Esta idea continúa presente en numerosos algoritmos modernos, donde varias rondas de sustituciones y permutaciones trabajan conjuntamente para producir un cifrado altamente resistente.

3.5 Criptoanálisis

El desarrollo de la criptografía estuvo acompañado desde sus orígenes por el nacimiento del criptoanálisis, disciplina dedicada al estudio de métodos para romper sistemas criptográficos sin conocer la llave secreta.

Mientras la criptografía busca proteger la información, el criptoanálisis intenta descubrir vulnerabilidades que permitan recuperar el mensaje original o deducir la llave utilizada durante el proceso de cifrado.

En la criptografía clásica, uno de los métodos más efectivos consistía en analizar la frecuencia de aparición de las letras. Debido a que cada idioma posee distribuciones estadísticas relativamente constantes, era posible asociar las letras más frecuentes del texto cifrado con las más comunes del lenguaje original.

Además del análisis estadístico, también podían aprovecharse patrones repetitivos, palabras conocidas o información parcial sobre el contenido del mensaje para facilitar el proceso de descifrado.

La competencia constante entre el diseño de nuevos algoritmos y el desarrollo de nuevas técnicas de ataque impulsó una evolución continua de la criptografía, fenómeno que continúa vigente en la actualidad.

3.6 La máquina Enigma

Uno de los sistemas criptográficos más importantes del siglo XX fue la máquina **Enigma**, utilizada principalmente por Alemania durante la Segunda Guerra Mundial para proteger sus comunicaciones militares.

A diferencia de los cifrados clásicos tradicionales, Enigma incorporaba componentes mecánicos y eléctricos que modificaban continuamente el proceso de sustitución después de cifrar cada letra. Este mecanismo producía un número extraordinariamente grande de configuraciones posibles, convirtiéndola en uno de los sistemas más avanzados de su época.

Cada pulsación sobre el teclado provocaba el movimiento de rotores internos, alterando automáticamente el alfabeto de sustitución utilizado para la siguiente letra. Como consecuencia, una misma letra podía cifrarse de formas completamente distintas dependiendo de su posición dentro del mensaje.

Durante varios años este sistema fue considerado prácticamente irrompible. Sin embargo, diversos equipos de criptógrafos desarrollaron métodos matemáticos y electromecánicos capaces de reducir enormemente el espacio de búsqueda de configuraciones posibles. Estos avances permitieron recuperar información estratégica de enorme importancia durante la guerra.

El caso de Enigma demostró que incluso sistemas extremadamente complejos pueden presentar debilidades si existen errores de diseño, configuraciones inadecuadas o patrones repetitivos en su utilización.

3.7 One-Time Pad

Entre todos los sistemas criptográficos conocidos, el **One-Time Pad** ocupa un lugar especial debido a que constituye el único algoritmo cuya seguridad perfecta ha sido demostrada matemáticamente bajo determinadas condiciones.

Su funcionamiento consiste en combinar cada símbolo del mensaje con una llave completamente aleatoria de la misma longitud

utilizando operaciones específicas de cifrado. Una vez utilizada, la llave debe destruirse y nunca volver a emplearse.

La seguridad del One-Time Pad depende de tres condiciones fundamentales:

- La llave debe ser completamente aleatoria.
- Debe tener exactamente la misma longitud que el mensaje.
- Debe utilizarse una sola vez.

Cuando estas condiciones se cumplen rigurosamente, el texto cifrado no proporciona ninguna información sobre el contenido original. Desde un punto de vista matemático, todos los mensajes posibles resultan igualmente probables, haciendo imposible determinar cuál corresponde al texto verdadero.

No obstante, este sistema presenta importantes dificultades prácticas. La necesidad de generar, distribuir, almacenar y proteger llaves tan largas como cada mensaje limita considerablemente su utilización en aplicaciones reales. Por esta razón, aunque continúa siendo un referente teórico de seguridad, la mayoría de los sistemas modernos emplean algoritmos diferentes que ofrecen un equilibrio más adecuado entre seguridad y eficiencia.

3.8 Limitaciones de la criptografía clásica

A pesar de su importancia histórica, la criptografía clásica presenta diversas limitaciones que impiden su utilización en sistemas informáticos modernos.

En primer lugar, la mayoría de estos algoritmos fueron diseñados para proteger mensajes escritos manualmente, no grandes volúmenes de información digital. Además, muchos de ellos pueden romperse mediante técnicas estadísticas relativamente sencillas, especialmente cuando el atacante dispone de una cantidad suficiente de texto cifrado.

Otra limitación importante radica en el reducido tamaño de sus espacios de búsqueda. Con el desarrollo de las computadoras modernas, numerosas técnicas clásicas pueden romperse mediante ataques por fuerza bruta en cuestión de segundos.

Finalmente, estos sistemas carecen de mecanismos para proporcionar autenticidad, integridad o intercambio seguro de llaves, propiedades consideradas indispensables en los sistemas de comunicación actuales.

Estas limitaciones impulsaron el desarrollo de la criptografía moderna, basada en problemas matemáticos cuya resolución resulta computacionalmente inviable con la tecnología disponible. A partir de esta transición surgieron algoritmos como DES, AES, RSA y los protocolos criptográficos utilizados actualmente para proteger las comunicaciones en Internet.

Capítulo 4. Criptografía Simétrica

4.1 Introducción

El desarrollo de las computadoras transformó por completo la manera en que la información debía protegerse. Los algoritmos clásicos, diseñados para operar sobre letras y palabras, dejaron de ser suficientes frente al incremento de la capacidad de procesamiento y la aparición de nuevas técnicas de criptoanálisis. Fue necesario diseñar sistemas capaces de trabajar directamente con información digital y ofrecer un nivel de seguridad considerablemente mayor.

La criptografía simétrica surge como respuesta a esta necesidad. En este paradigma, el emisor y el receptor comparten una misma llave secreta, utilizada tanto para cifrar como para descifrar la información. Mientras dicha llave permanezca protegida, el algoritmo garantiza que un tercero no pueda recuperar el contenido del mensaje en un tiempo razonable.

Actualmente, la criptografía simétrica constituye uno de los mecanismos más utilizados para proteger grandes cantidades de información debido a su elevada velocidad de procesamiento. Sistemas de almacenamiento, bases de datos, redes privadas virtuales, aplicaciones de mensajería y protocolos de comunicación emplean algoritmos simétricos para cifrar información de manera eficiente.

Sin embargo, la principal dificultad de este enfoque no reside en el algoritmo, sino en la distribución segura de la llave secreta. Si un atacante obtiene la llave utilizada por ambas partes, podrá descifrar todas las comunicaciones protegidas mediante ella. Este problema impulsó posteriormente el desarrollo de la criptografía de llave pública, aunque la criptografía simétrica continúa siendo indispensable debido a su eficiencia.

4.2 Características de la criptografía simétrica

La principal característica de este tipo de algoritmos es el empleo de una única llave para realizar tanto el cifrado como el descifrado de la información.

Cuando un usuario desea enviar un mensaje confidencial, aplica el algoritmo de cifrado utilizando una llave previamente compartida con el receptor. El resultado es un texto cifrado que puede transmitirse por un canal inseguro sin revelar el contenido original. Una vez recibido, el destinatario utiliza exactamente la misma llave para recuperar el mensaje inicial.

Esta simplicidad ofrece importantes ventajas desde el punto de vista computacional. Los algoritmos simétricos requieren una cantidad relativamente pequeña de operaciones matemáticas, permitiendo procesar grandes volúmenes de datos en tiempos muy reducidos.

Entre sus principales ventajas destacan:

- Alta velocidad de cifrado y descifrado.
- Bajo consumo de recursos computacionales.

- Adecuados para proteger archivos de gran tamaño.
- Implementación eficiente tanto en software como en hardware.

No obstante, también presentan limitaciones importantes. La necesidad de compartir previamente una llave secreta representa el principal desafío de este tipo de sistemas. En entornos donde los participantes nunca han tenido contacto previo, establecer dicha llave de forma segura puede resultar complejo.

Por esta razón, los sistemas modernos suelen combinar criptografía simétrica y criptografía asimétrica. Mientras los algoritmos de llave pública permiten intercambiar de manera segura una llave secreta, la criptografía simétrica se encarga posteriormente de proteger toda la comunicación debido a su mayor eficiencia.

4.3 Cifrados de bloque

Los algoritmos simétricos pueden clasificarse en dos grandes categorías: cifrados de flujo y cifrados de bloque.

Los cifrados de bloque procesan la información dividiéndola en bloques de tamaño fijo. Cada bloque es transformado mediante una serie de operaciones matemáticas controladas por la llave secreta. Una vez cifrado un bloque, el algoritmo continúa con el siguiente hasta completar todo el mensaje.

Este enfoque presenta importantes ventajas desde el punto de vista del diseño criptográfico. Al trabajar sobre bloques de tamaño conocido, resulta posible aplicar múltiples rondas de transformación capaces de incrementar significativamente la resistencia frente a ataques.

La mayoría de los algoritmos modernos pertenecen a esta categoría. Entre ellos destacan DES, Triple DES y, especialmente, AES, considerado actualmente el estándar internacional para el cifrado simétrico.

4.4 Principios de diseño

Un algoritmo criptográfico moderno no basa su seguridad en mantener en secreto su funcionamiento interno. De acuerdo con el principio de Kerckhoffs, ampliamente aceptado en criptografía, la seguridad de un sistema debe depender únicamente del secreto de la llave.

Esto implica que cualquier investigador puede conocer completamente el algoritmo sin comprometer la seguridad de las comunicaciones. Incluso si un atacante dispone del código fuente, la documentación técnica y una descripción detallada del funcionamiento interno, seguirá siendo incapaz de recuperar la información siempre que desconozca la llave utilizada.

Para cumplir este principio, los algoritmos modernos incorporan múltiples transformaciones destinadas a producir dos propiedades fundamentales:

Confusión

La confusión busca dificultar la relación entre la llave y el texto cifrado. Un pequeño cambio en la llave debe modificar completamente el resultado del cifrado, impidiendo deducir información útil sobre la llave observando únicamente los mensajes cifrados.

Difusión

La difusión pretende distribuir la información del texto original a lo largo de todo el bloque cifrado. De esta manera, modificar un solo bit del mensaje original produce cambios aparentemente aleatorios en numerosos bits del texto cifrado.

Estas dos propiedades fueron propuestas por Claude Shannon y continúan siendo uno de los pilares fundamentales del diseño criptográfico moderno.

4.5 Redes de sustitución y permutación

Muchos algoritmos modernos, incluido AES, utilizan una estructura conocida como **Red de Sustitución-Permutación**.

Esta arquitectura consiste en aplicar repetidamente dos tipos de transformaciones:

- Sustituciones.
- Permutaciones.

Las sustituciones reemplazan pequeños grupos de bits mediante tablas matemáticamente diseñadas denominadas **S-Boxes**. Estas transformaciones introducen no linealidad en el algoritmo, dificultando enormemente el análisis matemático del cifrado.

Las permutaciones reorganizan la posición de los bits dentro del bloque. Aunque por sí mismas no incrementan la seguridad, permiten que la información modificada durante una ronda afecte progresivamente al resto del bloque en las rondas siguientes.

Después de varias iteraciones de sustituciones y permutaciones, el texto cifrado presenta una apariencia completamente aleatoria. Esta estructura proporciona una elevada resistencia frente a numerosos ataques criptográficos conocidos.

4.6 El algoritmo DES

Durante varias décadas, el estándar más importante para el cifrado simétrico fue el **Data Encryption Standard (DES)**.

DES fue desarrollado durante la década de 1970 con el objetivo de establecer un algoritmo de uso general para proteger información sensible. Su diseño se basa en una red de Feistel que divide cada bloque de información en dos mitades y aplica múltiples rondas de transformación controladas por una llave secreta.

El algoritmo trabaja con bloques de **64 bits** y utiliza una llave efectiva de **56 bits**. Cada bloque atraviesa dieciséis rondas consecutivas de procesamiento antes de producir el texto cifrado final.

Durante muchos años DES ofreció un nivel adecuado de seguridad. Sin embargo, el constante aumento de la capacidad computacional permitió realizar ataques por fuerza bruta en tiempos cada vez menores. Debido al reducido tamaño de su llave, un atacante moderno puede probar todas las combinaciones posibles utilizando hardware especializado.

Como consecuencia, DES dejó de considerarse seguro para aplicaciones actuales.

4.7 Triple DES

Para prolongar la vida útil de DES sin modificar completamente su estructura se desarrolló Triple DES, también conocido como 3DES.

En lugar de cifrar la información una única vez, Triple DES aplica el algoritmo DES en tres ocasiones consecutivas utilizando dos o tres llaves diferentes.

Este procedimiento incrementa considerablemente la seguridad al aumentar el espacio de búsqueda que debería explorar un atacante. Sin embargo, también triplica aproximadamente el tiempo necesario para procesar la información.

Aunque Triple DES representó una solución temporal ampliamente utilizada durante varios años, finalmente fue reemplazado por AES debido a sus mejores características de seguridad y rendimiento.

4.8 Nacimiento del AES

A finales del siglo XX resultaba evidente que era necesario desarrollar un nuevo estándar criptográfico capaz de sustituir definitivamente a DES.

Para ello se organizó un proceso internacional en el que diversos equipos de investigación presentaron propuestas de nuevos algoritmos criptográficos. Tras varios años de evaluación, análisis públicos y pruebas de resistencia frente a diferentes ataques, fue seleccionado el algoritmo **Rijndael**, desarrollado por los criptógrafos belgas Joan Daemen y Vincent Rijmen.

Posteriormente, este algoritmo fue adoptado oficialmente como el **Advanced Encryption Standard (AES)**.

AES fue diseñado para ofrecer un alto nivel de seguridad, gran eficiencia computacional y facilidad de implementación tanto en software como en hardware. Gracias a estas características, actualmente constituye el algoritmo de cifrado simétrico más utilizado en el mundo.

Su adopción se extiende a prácticamente todos los ámbitos de la informática moderna, incluyendo redes inalámbricas, almacenamiento de información, dispositivos móviles, sistemas bancarios, comercio electrónico y protocolos de comunicación seguros como TLS.

4.9 Advanced Encryption Standard (AES)

El **Advanced Encryption Standard (AES)** constituye actualmente el algoritmo de cifrado simétrico más utilizado a nivel mundial. Desde su adopción como estándar internacional, se ha convertido en el mecanismo principal para proteger información en aplicaciones comerciales, gubernamentales y militares. Su éxito radica en la combinación de un elevado nivel de seguridad, una implementación eficiente y una gran flexibilidad para adaptarse a diferentes plataformas tecnológicas.

AES fue seleccionado tras un proceso internacional de evaluación en el que participaron numerosos algoritmos candidatos. El objetivo consistía en reemplazar definitivamente a DES mediante un sistema capaz de resistir los avances tecnológicos y los nuevos métodos de ataque desarrollados por la comunidad criptográfica. Después de varios años de análisis públicos, el algoritmo Rijndael fue elegido como nuevo estándar debido a su robustez matemática y a su excelente desempeño.

A diferencia de DES, cuya estructura se basa en una red de Feistel, AES utiliza una **Red de Sustitución-Permutación (Substitution-Permutation Network)**. Esta arquitectura aplica repetidamente transformaciones matemáticas sobre bloques de

información, logrando que cada ronda incremente progresivamente la complejidad del texto cifrado.

El algoritmo trabaja siempre con bloques de **128 bits**, equivalentes a dieciséis bytes. No obstante, permite utilizar llaves de diferentes tamaños: **128, 192 o 256 bits**. El tamaño de la llave determina el número de rondas que ejecutará el algoritmo durante el proceso de cifrado. AES-128 utiliza diez rondas, AES-192 emplea doce y AES-256 realiza catorce rondas. A medida que aumenta el número de rondas también se incrementa la resistencia frente a ataques criptográficos, aunque ello implica un ligero aumento en el tiempo de procesamiento.

4.10 Estructura interna de AES

Antes de comenzar las rondas de cifrado, el bloque de información se organiza en una estructura matricial conocida como **estado** (*State*). Esta matriz está formada por cuatro filas y cuatro columnas, donde cada posición almacena un byte del bloque original.

La representación matricial facilita la aplicación de las distintas transformaciones del algoritmo. Cada ronda opera directamente sobre esta matriz, modificando progresivamente sus valores hasta obtener el texto cifrado final.

Paralelamente, la llave secreta también es procesada mediante un algoritmo denominado **expansión de llave** (*Key Expansion*). Este procedimiento genera una serie de subllaves diferentes, una para cada ronda del algoritmo. El uso de múltiples subllaves impide que todas las transformaciones dependan exactamente del mismo conjunto de bits, incrementando considerablemente la complejidad del sistema.

4.11 Expansión de llave

Aunque el usuario únicamente proporciona una llave inicial, AES requiere una llave distinta para cada una de las rondas que

ejecuta durante el proceso de cifrado. Para obtenerlas se utiliza un procedimiento denominado expansión de llave.

Este algoritmo genera nuevas subllaves aplicando operaciones de rotación, sustitución mediante la S-Box y combinaciones con constantes previamente definidas. Como resultado, cada ronda utiliza una versión diferente de la llave original.

La expansión de llave cumple dos objetivos fundamentales. En primer lugar, evita que una misma llave sea utilizada repetidamente durante todas las rondas del algoritmo. En segundo lugar, introduce relaciones matemáticas suficientemente complejas para dificultar el desarrollo de ataques dirigidos contra la estructura interna de AES.

La seguridad del algoritmo depende en gran medida de este proceso, ya que cualquier debilidad en la generación de subllaves podría facilitar el descubrimiento de la llave principal.

4.12 Transformación SubBytes

La primera transformación aplicada durante cada ronda recibe el nombre de **SubBytes**.

En esta etapa, cada byte del estado es sustituido por otro valor obtenido a partir de una tabla denominada **S-Box** (*Substitution Box*). Esta tabla no ha sido construida de manera arbitraria, sino mediante principios matemáticos cuidadosamente seleccionados para ofrecer resistencia frente a diferentes técnicas de criptoanálisis.

La S-Box constituye el principal componente no lineal de AES. Gracias a ella, la relación entre el texto plano, la llave y el texto cifrado resulta extremadamente difícil de modelar mediante expresiones matemáticas sencillas.

Desde el punto de vista criptográfico, esta transformación proporciona la propiedad de **confusión**, ya que dificulta enormemente establecer relaciones directas entre los datos de entrada y los resultados obtenidos.

Una característica importante es que la sustitución siempre puede invertirse utilizando una tabla inversa. Esto permite que el proceso de descifrado recupere exactamente los valores originales.

4.13 Transformación ShiftRows

Después de sustituir los bytes del estado, AES aplica una segunda transformación denominada **ShiftRows**.

En esta etapa no se modifican los valores almacenados en la matriz. Únicamente cambia su posición mediante desplazamientos circulares realizados sobre cada fila.

La primera fila permanece sin cambios.

La segunda fila se desplaza una posición hacia la izquierda.

La tercera fila se desplaza dos posiciones.

Finalmente, la cuarta fila se desplaza tres posiciones.

Este procedimiento distribuye progresivamente la información por toda la matriz, evitando que los bytes relacionados permanezcan agrupados en las mismas columnas.

Aunque esta operación parece sencilla, resulta fundamental para que los cambios introducidos durante una ronda afecten posteriormente a un mayor número de posiciones.

4.14 Transformación MixColumns

Una vez reorganizadas las filas, el algoritmo aplica una tercera transformación denominada **MixColumns**.

En esta etapa cada columna del estado se considera un vector matemático que es multiplicado por una matriz constante dentro de un cuerpo finito. El resultado consiste en reemplazar completamente los cuatro bytes originales por nuevos valores obtenidos mediante operaciones algebraicas.

El propósito principal de esta transformación consiste en incrementar la difusión del algoritmo. Después de MixColumns,

cada byte depende simultáneamente de varios bytes anteriores, provocando que cualquier modificación realizada sobre el texto plano se propague rápidamente a lo largo de todo el bloque.

Esta característica dificulta enormemente los ataques diferenciales y lineales desarrollados para analizar algoritmos criptográficos.

Es importante señalar que la última ronda de AES omite esta transformación. Todas las rondas anteriores incluyen MixColumns, pero la ronda final únicamente ejecuta SubBytes, ShiftRows y AddRoundKey.

4.15 Transformación AddRoundKey

La última operación de cada ronda consiste en combinar el estado con la subllave correspondiente mediante una operación XOR.

Esta transformación recibe el nombre de **AddRoundKey** y representa el único momento en que la llave secreta interviene directamente sobre los datos durante cada ronda.

Cada byte del estado se combina con el byte correspondiente de la subllave. Debido a las propiedades de la operación XOR, el proceso puede invertirse fácilmente durante el descifrado utilizando exactamente la misma subllave.

Aunque matemáticamente resulta una operación muy simple, su combinación con las transformaciones anteriores proporciona un elevado nivel de seguridad. La información ha sido previamente sustituida, desplazada y mezclada, por lo que la incorporación de la llave modifica completamente el resultado obtenido.

4.16 Desarrollo completo de una ronda

Cada ronda de AES sigue siempre el mismo orden de ejecución:

1. Sustitución de bytes mediante SubBytes.
2. Desplazamiento de filas utilizando ShiftRows.
3. Mezcla de columnas mediante MixColumns.

4. Combinación con la subllave utilizando AddRoundKey.

Antes de comenzar la primera ronda existe una operación inicial AddRoundKey realizada directamente sobre el texto plano.

Posteriormente se ejecutan todas las rondas definidas por el tamaño de la llave.

Finalmente, la última ronda elimina MixColumns y concluye con la generación del texto cifrado.

Gracias a la repetición de este conjunto de transformaciones, incluso una modificación mínima en el mensaje original produce un resultado completamente diferente. Este comportamiento recibe el nombre de **efecto avalancha** y constituye una de las propiedades más importantes de los algoritmos criptográficos modernos.

4.17 Seguridad de AES

Desde su adopción como estándar internacional, AES ha sido sometido a un análisis constante por parte de investigadores de todo el mundo. A diferencia de algoritmos anteriores, su diseño fue completamente público, permitiendo que la comunidad científica evaluara exhaustivamente su resistencia frente a diversas técnicas de ataque.

Hasta la fecha, no se conocen ataques prácticos capaces de comprometer el algoritmo cuando se implementa correctamente y se utilizan tamaños de llave adecuados. La seguridad de AES no depende del secreto de su funcionamiento interno, sino de la enorme complejidad computacional asociada a la búsqueda de la llave correcta.

Por esta razón, AES continúa siendo el algoritmo recomendado para proteger información sensible en aplicaciones civiles, comerciales y gubernamentales.

4.18 Modos de Operación de AES

Como se explicó anteriormente, AES trabaja exclusivamente con bloques de **128 bits**. Esta característica implica que únicamente

puede cifrar una cantidad fija de información durante cada ejecución del algoritmo. Sin embargo, en aplicaciones reales es común que los mensajes tengan tamaños variables, desde unos cuantos bytes hasta archivos de varios gigabytes.

Para resolver esta limitación se desarrollaron los **modos de operación**, mecanismos que indican la forma en que deben procesarse múltiples bloques consecutivos utilizando un mismo algoritmo de cifrado. Estos modos no modifican el funcionamiento interno de AES; simplemente establecen cómo deben relacionarse entre sí los diferentes bloques de información durante el proceso de cifrado y descifrado.

La elección del modo de operación tiene un impacto directo sobre la seguridad del sistema. Dos implementaciones que utilicen exactamente el mismo algoritmo AES pueden ofrecer niveles de protección muy diferentes dependiendo del modo empleado. Por esta razón, comprender sus características resulta tan importante como conocer el funcionamiento interno del algoritmo.

Los modos de operación más utilizados son ECB, CBC, CFB, OFB, CTR y GCM. Cada uno fue diseñado para resolver necesidades específicas relacionadas con la confidencialidad, la integridad, la eficiencia y la tolerancia a errores.

4.19 Electronic Codebook (ECB)

El modo **Electronic Codebook (ECB)** representa la forma más sencilla de utilizar un cifrado de bloque. En este esquema, cada bloque del mensaje se cifra de manera completamente independiente utilizando la misma llave.

Si dos bloques del texto plano contienen exactamente la misma información, ambos producirán el mismo bloque cifrado. Desde el punto de vista matemático, el algoritmo funciona correctamente; sin embargo, esta propiedad introduce una importante vulnerabilidad.

Cuando un archivo contiene regiones repetitivas, dichas repeticiones permanecen visibles después del cifrado. En imágenes

digitales, por ejemplo, es posible identificar parcialmente las formas originales aun cuando todos los datos hayan sido cifrados. Esto demuestra que ECB no oculta adecuadamente los patrones presentes en el mensaje.

La principal ventaja de este modo radica en su simplicidad y en la posibilidad de cifrar cada bloque de forma independiente, permitiendo un procesamiento altamente paralelo. No obstante, debido a las debilidades mencionadas, ECB no se considera adecuado para proteger información sensible y su uso se limita a situaciones muy específicas donde la repetición de bloques no representa un problema.

4.20 Cipher Block Chaining (CBC)

Con el propósito de eliminar los patrones visibles presentes en ECB se desarrolló el modo **Cipher Block Chaining (CBC)**.

En este esquema, cada bloque del mensaje se combina mediante una operación XOR con el bloque cifrado anterior antes de ser procesado por AES. Como consecuencia, el resultado obtenido para un bloque depende no solamente de su contenido, sino también de todos los bloques procesados previamente.

El primer bloque constituye un caso especial, ya que no existe un bloque cifrado anterior. Para resolver esta situación se utiliza un **Vector de Inicialización (Initialization Vector o IV)**, un valor aleatorio que actúa como punto de partida para la cadena de cifrado.

El empleo del IV garantiza que incluso dos mensajes idénticos produzcan resultados completamente diferentes cuando se cifran en momentos distintos.

Gracias a esta característica, CBC elimina la repetición de patrones observada en ECB y proporciona un nivel considerablemente mayor de confidencialidad.

Sin embargo, este modo también presenta ciertas limitaciones. Debido a que cada bloque depende del anterior, el proceso de

descifrado requiere seguir exactamente el orden original de los bloques. Además, un error introducido durante la transmisión afecta tanto al bloque correspondiente como al siguiente, propagando parcialmente las modificaciones.

Durante muchos años CBC fue uno de los modos más utilizados para proteger comunicaciones y archivos. Actualmente continúa siendo seguro cuando se implementa correctamente, aunque en numerosas aplicaciones ha sido reemplazado por modos más modernos.

4.21 Cipher Feedback (CFB)

El modo **Cipher Feedback (CFB)** transforma un cifrado de bloque en un mecanismo capaz de procesar información de forma continua, aproximándose al comportamiento de un cifrado de flujo.

En lugar de cifrar directamente el bloque del mensaje, AES cifra inicialmente un registro interno. El resultado obtenido se combina posteriormente con el texto plano mediante una operación XOR para producir el texto cifrado.

Después de cada bloque, parte del resultado generado se reutiliza como entrada para la siguiente operación, creando una dependencia continua entre los diferentes bloques del mensaje.

Una de las principales ventajas de CFB consiste en que puede trabajar con unidades menores que un bloque completo, permitiendo cifrar datos conforme son generados. Esta característica resulta útil en aplicaciones donde la información se transmite de manera continua, como ciertos sistemas de comunicación en tiempo real.

No obstante, la dependencia existente entre bloques implica que un error durante la transmisión puede propagarse parcialmente antes de desaparecer.

4.22 Output Feedback (OFB)

El modo **Output Feedback (OFB)** también convierte un cifrado de bloque en un cifrado de flujo, aunque utiliza un mecanismo diferente.

En este caso, AES nunca procesa directamente el texto plano. En su lugar, genera una secuencia pseudoaleatoria de bits a partir del vector de inicialización y de los resultados obtenidos en iteraciones anteriores. Posteriormente, esta secuencia se combina mediante XOR con el mensaje para producir el texto cifrado.

Una característica importante de OFB es que la secuencia generada depende exclusivamente de la llave y del vector de inicialización. Como consecuencia, los errores de transmisión no se propagan entre bloques, ya que cada posición del mensaje únicamente depende del correspondiente fragmento de la secuencia generada.

Esta propiedad convierte a OFB en una alternativa adecuada para canales de comunicación donde pueden presentarse errores ocasionales durante la transmisión.

Sin embargo, reutilizar un mismo vector de inicialización con la misma llave compromete gravemente la seguridad del sistema, por lo que cada cifrado debe emplear un IV diferente.

4.23 Counter Mode (CTR)

El modo **Counter (CTR)** constituye una de las alternativas más eficientes para utilizar AES.

Su funcionamiento se basa en el cifrado de una secuencia de contadores consecutivos. Cada contador se cifra utilizando AES y el resultado obtenido se combina mediante XOR con el bloque correspondiente del mensaje.

A diferencia de CBC, los bloques son completamente independientes entre sí, permitiendo cifrar y descifrar múltiples bloques simultáneamente. Esta característica facilita enormemente el procesamiento paralelo y mejora el rendimiento en sistemas con varios núcleos de procesamiento.

Otra ventaja importante consiste en que cualquier bloque puede descifrarse directamente sin necesidad de procesar todos los

bloques anteriores, simplificando el acceso aleatorio a la información cifrada.

CTR ofrece un elevado nivel de eficiencia y actualmente constituye uno de los modos preferidos en numerosas aplicaciones modernas.

No obstante, resulta absolutamente indispensable que un mismo contador nunca sea reutilizado junto con la misma llave. Si esta condición se viola, la seguridad del sistema puede verse seriamente comprometida.

4.24 Galois/Counter Mode (GCM)

El modo **Galois/Counter Mode (GCM)** representa una evolución del modo CTR diseñada para proporcionar simultáneamente **confidencialidad e integridad**.

Mientras los modos anteriores únicamente protegen el contenido del mensaje, GCM incorpora además un mecanismo capaz de verificar que la información no haya sido modificada durante la transmisión.

Para lograrlo combina el cifrado mediante Counter Mode con un algoritmo de autenticación basado en operaciones realizadas dentro de un cuerpo de Galois. El resultado final incluye tanto el texto cifrado como una etiqueta de autenticación (*Authentication Tag*).

Durante el proceso de descifrado, el receptor verifica primero dicha etiqueta. Si detecta cualquier modificación, el mensaje completo es rechazado antes de intentar recuperar el contenido original.

Esta característica protege el sistema frente a numerosos ataques donde un adversario intenta alterar información cifrada sin conocer la llave.

Debido a que proporciona confidencialidad, integridad y autenticación de manera simultánea, GCM se ha convertido en el modo recomendado para la mayoría de las aplicaciones modernas,

incluyendo el protocolo **TLS 1.3**, utilizado actualmente para proteger las comunicaciones en Internet.

4.25 Comparación entre los modos de operación

Cada modo de operación presenta ventajas y limitaciones particulares, por lo que la elección depende de las necesidades específicas de la aplicación.

ECB destaca por su simplicidad y rapidez, pero revela patrones presentes en el mensaje, razón por la cual no resulta apropiado para proteger información confidencial.

CBC elimina estos patrones mediante el encadenamiento de bloques, proporcionando un mayor nivel de seguridad a costa de un procesamiento secuencial.

CFB y OFB convierten el algoritmo en un cifrado de flujo, facilitando la transmisión continua de datos y ofreciendo distintos comportamientos frente a errores durante la comunicación.

CTR proporciona un excelente rendimiento gracias a la posibilidad de procesar múltiples bloques simultáneamente, convirtiéndose en una opción ampliamente utilizada en aplicaciones de alto desempeño.

Finalmente, GCM combina las ventajas de CTR con mecanismos de autenticación que garantizan tanto la confidencialidad como la integridad de la información, motivo por el cual constituye actualmente una de las alternativas más seguras para la protección de comunicaciones digitales.

Capítulo 5. Funciones Hash, Códigos de Autenticación y Derivación de Llaves

5.1 Introducción

Hasta este punto se ha estudiado cómo la criptografía simétrica permite proteger la confidencialidad de la información mediante

algoritmos de cifrado como AES. Sin embargo, garantizar que un tercero no pueda leer un mensaje no es suficiente para asegurar completamente una comunicación. También es necesario verificar que los datos recibidos no hayan sido modificados durante su transmisión y confirmar que realmente provienen del emisor esperado.

La criptografía moderna aborda estos problemas mediante mecanismos especializados como las funciones hash, los códigos de autenticación de mensajes (MAC), los códigos HMAC y los algoritmos de derivación de llaves. Aunque estos mecanismos no cifran la información, desempeñan un papel fundamental en la protección de los sistemas informáticos actuales.

Las funciones hash permiten detectar modificaciones accidentales o maliciosas en un conjunto de datos, mientras que los códigos de autenticación añaden la capacidad de verificar el origen de la información. Por su parte, los algoritmos de derivación de llaves generan nuevas llaves criptográficas a partir de un secreto inicial, fortaleciendo la seguridad de numerosos protocolos.

Actualmente, prácticamente todos los protocolos de seguridad combinan algoritmos de cifrado con estos mecanismos de autenticidad e integridad. Sin ellos, sería posible alterar mensajes cifrados o suplantar la identidad de los participantes sin ser detectado.

5.2 Concepto de función hash

Una **función hash criptográfica** es un algoritmo matemático que transforma una entrada de longitud arbitraria en una salida de longitud fija denominada **valor hash**, **resumen criptográfico** o **digest**.

La característica más importante de una función hash es que siempre produce un resultado del mismo tamaño, independientemente de la cantidad de información procesada. Un archivo de algunos kilobytes y otro de varios gigabytes generan resúmenes de igual longitud cuando se utiliza el mismo algoritmo hash.

Este resumen puede considerarse una especie de "huella digital" del mensaje. Cualquier modificación realizada sobre la información original, incluso el cambio de un único bit, produce un valor hash completamente diferente.

Gracias a esta propiedad, las funciones hash permiten verificar fácilmente si un archivo ha sido alterado. Basta con calcular nuevamente el resumen y compararlo con el valor obtenido anteriormente. Si ambos coinciden, existe una alta probabilidad de que la información permanezca intacta.

Es importante destacar que una función hash **no cifra información**. El objetivo no consiste en ocultar el contenido del mensaje, sino en generar un identificador matemático que permita detectar modificaciones.

5.3 Propiedades criptográficas de una función hash

Para que una función hash sea considerada segura debe satisfacer una serie de propiedades matemáticas que dificulten cualquier intento de manipulación.

La primera propiedad es el **determinismo**. Una misma entrada debe producir siempre exactamente el mismo resumen. Esta característica permite verificar la integridad de un archivo en cualquier momento.

La segunda propiedad corresponde a la **rapidez de cálculo**. Incluso para archivos muy grandes, el algoritmo debe generar el resumen en un tiempo reducido.

Otra propiedad fundamental es la **resistencia a la preimagen**. Dado únicamente un valor hash, debe resultar computacionalmente inviable reconstruir el mensaje original que lo produjo.

También se requiere **resistencia a la segunda preimagen**. Si se conoce un mensaje específico, no debe ser posible encontrar otro mensaje diferente que produzca exactamente el mismo resumen.

Finalmente, la propiedad más importante es la **resistencia a colisiones**. Debe resultar extremadamente difícil encontrar dos

mensajes distintos cuyo valor hash sea idéntico.

Estas propiedades garantizan que el resumen criptográfico pueda utilizarse como un identificador prácticamente único para cada conjunto de datos.

5.4 Efecto avalancha

Una de las características más representativas de las funciones hash modernas es el **efecto avalancha**.

Este fenómeno establece que una modificación mínima en el mensaje original debe provocar un cambio completamente diferente en el resumen generado. Incluso alterar un único bit produce un valor hash aparentemente aleatorio y sin relación visible con el anterior.

El efecto avalancha dificulta enormemente cualquier intento de deducir información sobre el mensaje original observando únicamente los resúmenes generados.

Además, esta propiedad permite detectar inmediatamente cualquier alteración realizada sobre un archivo, por pequeña que sea.

5.5 Algoritmos hash

A lo largo de la evolución de la criptografía se han desarrollado numerosos algoritmos hash con diferentes niveles de seguridad.

Entre los primeros algoritmos ampliamente utilizados se encuentran **MD5** y **SHA-1**. Durante varios años fueron considerados suficientemente seguros para múltiples aplicaciones. Sin embargo, los avances en criptoanálisis permitieron descubrir vulnerabilidades que facilitan la generación de colisiones, reduciendo considerablemente su confiabilidad.

Como consecuencia, ambos algoritmos dejaron de recomendarse para aplicaciones donde la seguridad constituye un requisito fundamental.

Posteriormente se desarrolló la familia **SHA-2**, integrada por diferentes variantes como SHA-224, SHA-256, SHA-384 y SHA-512.

Estos algoritmos ofrecen un nivel de seguridad considerablemente superior y continúan siendo ampliamente utilizados en aplicaciones modernas.

En la actualidad también existe la familia **SHA-3**, diseñada con una arquitectura diferente que proporciona una alternativa adicional para aplicaciones criptográficas de alta seguridad.

5.6 Aplicaciones de las funciones hash

Las funciones hash poseen una enorme variedad de aplicaciones dentro de la seguridad informática.

Una de las más importantes consiste en la **verificación de integridad de archivos**. Cuando un documento es distribuido por Internet, puede publicarse junto con su correspondiente valor hash. El receptor calcula nuevamente el resumen del archivo descargado y lo compara con el proporcionado por el emisor. Si ambos coinciden, es muy probable que el archivo no haya sido modificado.

Otra aplicación importante corresponde al almacenamiento seguro de contraseñas. En lugar de guardar directamente la contraseña del usuario, los sistemas almacenan únicamente su valor hash. Durante el proceso de autenticación, la contraseña ingresada vuelve a procesarse mediante la misma función y el resultado obtenido se compara con el resumen almacenado.

Este procedimiento evita que las contraseñas originales queden registradas directamente dentro de la base de datos.

Las funciones hash también forman parte de las firmas digitales, los certificados electrónicos, las cadenas de bloques y numerosos protocolos de comunicación segura.

5.7 Limitaciones de las funciones hash

Aunque las funciones hash permiten detectar modificaciones sobre la información, presentan una limitación importante: **no autentican al emisor**.

Un atacante podría modificar un archivo y calcular posteriormente un nuevo resumen utilizando exactamente el mismo algoritmo. Si el receptor desconoce cuál era el hash original, no tendría forma de distinguir cuál de los dos fue generado por el propietario legítimo.

En consecuencia, las funciones hash garantizan únicamente la integridad de los datos cuando existe un mecanismo confiable para distribuir el valor hash correspondiente.

Para resolver este problema surgieron los **Códigos de Autenticación de Mensajes**, conocidos como MAC.

5.8 Message Authentication Code (MAC)

Un **Message Authentication Code (MAC)** es un mecanismo criptográfico que combina una función hash con una **llave secreta compartida** entre el emisor y el receptor.

A diferencia de una función hash convencional, el cálculo del MAC depende tanto del contenido del mensaje como de una llave secreta. Esto significa que únicamente quienes conocen dicha llave pueden generar correctamente el código de autenticación.

Cuando el receptor recibe un mensaje, calcula nuevamente el MAC utilizando la misma llave compartida. Si el resultado coincide con el valor recibido, puede concluir que el mensaje no ha sido modificado y que probablemente proviene del participante legítimo.

De esta forma, el MAC proporciona simultáneamente **integridad y autenticidad**.

Sin embargo, al compartir ambos usuarios la misma llave, ninguno de ellos puede demostrar posteriormente cuál fue el verdadero autor del mensaje. Por esta razón, los MAC no proporcionan la propiedad de **no repudio**, característica que será abordada más adelante mediante las firmas digitales.

5.9 HMAC (Hash-based Message Authentication Code)

Aunque los códigos MAC ofrecen un mecanismo eficaz para verificar la autenticidad de un mensaje, existen diferentes formas de implementarlos. Una de las más utilizadas en la actualidad es **HMAC (Hash-based Message Authentication Code)**, un mecanismo que combina una función hash criptográfica con una llave secreta compartida.

El objetivo principal de HMAC consiste en aprovechar la robustez de algoritmos como SHA-256 o SHA-512 para generar un código de autenticación resistente a ataques criptográficos. A diferencia de calcular simplemente el hash del mensaje junto con la llave, HMAC aplica una estructura específica de operaciones diseñada para evitar diversas vulnerabilidades conocidas.

El funcionamiento general puede resumirse en tres etapas. En primer lugar, la llave secreta es adaptada al tamaño requerido por la función hash utilizada. Posteriormente, se combina con el mensaje mediante operaciones matemáticas previamente definidas y se calcula un primer resumen criptográfico. Finalmente, este resultado vuelve a procesarse junto con la llave para obtener el código de autenticación definitivo.

Aunque el procedimiento interno pueda parecer complejo, su utilización resulta transparente para el usuario. Las bibliotecas criptográficas implementan automáticamente todas estas operaciones, permitiendo que el desarrollador utilice HMAC como una única función.

Gracias a su diseño, HMAC mantiene un elevado nivel de seguridad incluso cuando la función hash subyacente presenta determinadas debilidades parciales. Esta característica explica por qué continúa siendo ampliamente utilizado en protocolos de comunicación, sistemas de autenticación y mecanismos de verificación de integridad.

5.10 Propiedades de HMAC

La principal ventaja de HMAC radica en que proporciona simultáneamente dos propiedades fundamentales: **integridad** y

autenticidad.

La integridad garantiza que el contenido del mensaje no haya sido alterado desde que fue generado por el emisor. Si un atacante modifica incluso un solo bit de la información, el código HMAC calculado por el receptor será completamente diferente y la alteración será detectada inmediatamente.

La autenticidad se obtiene gracias al uso de una llave secreta compartida. Únicamente quienes conocen dicha llave pueden generar correctamente el código HMAC correspondiente al mensaje. Esto permite verificar que la información proviene de un participante autorizado.

Otra propiedad importante consiste en la resistencia frente a ataques de extensión de longitud (*Length Extension Attacks*), una vulnerabilidad presente en determinadas funciones hash cuando se utilizan de manera incorrecta. La estructura específica de HMAC elimina este problema y proporciona una protección considerablemente mayor.

Por estas razones, HMAC constituye actualmente uno de los mecanismos preferidos para autenticar mensajes dentro de protocolos criptográficos modernos.

5.11 Aplicaciones de HMAC

El uso de HMAC se encuentra ampliamente extendido en aplicaciones donde resulta indispensable garantizar tanto la autenticidad como la integridad de la información.

Uno de sus principales ámbitos de aplicación corresponde a los protocolos de comunicación segura, donde permite verificar que los mensajes intercambiados entre dos participantes no hayan sido modificados durante la transmisión.

También se utiliza en sistemas de autenticación basados en tokens, servicios web, almacenamiento seguro de información, redes privadas virtuales y numerosas aplicaciones distribuidas.

En muchos protocolos, HMAC trabaja conjuntamente con algoritmos de cifrado simétrico. Mientras el cifrado protege la confidencialidad de los datos, HMAC verifica que dichos datos permanezcan íntegros y provengan realmente del emisor autorizado. Esta combinación constituye una estrategia ampliamente adoptada antes de la aparición de modos autenticados como AES-GCM, los cuales integran ambas funciones dentro de un único algoritmo.

5.12 Derivación de llaves (Key Derivation Functions)

En numerosos sistemas criptográficos no resulta conveniente utilizar directamente una contraseña o una llave maestra para cifrar información. En su lugar, se emplean mecanismos especializados capaces de generar nuevas llaves criptográficas a partir de un secreto inicial.

Estos mecanismos reciben el nombre de **Funciones de Derivación de Llaves (Key Derivation Functions o KDF)**.

Una KDF toma como entrada una contraseña, una llave compartida u otro secreto criptográfico y produce una o varias llaves independientes con características adecuadas para diferentes aplicaciones.

Este procedimiento ofrece varias ventajas. En primer lugar, evita reutilizar directamente el mismo secreto en múltiples algoritmos. Además, permite generar llaves de diferentes tamaños y propósitos sin comprometer la seguridad del secreto original.

Las funciones de derivación desempeñan un papel esencial en protocolos modernos como TLS, donde a partir de un único secreto compartido se generan múltiples llaves destinadas al cifrado, autenticación y protección de diferentes direcciones de comunicación.

5.13 Uso de Salt

Cuando las funciones hash se utilizan para proteger contraseñas, surge un problema importante. Si dos usuarios eligen exactamente la misma contraseña, ambos producirán el mismo valor hash.

Esta situación facilita ataques basados en tablas precalculadas, conocidas como **Rainbow Tables**, donde millones de contraseñas comunes ya han sido procesadas previamente mediante diferentes funciones hash.

Para evitar este problema se introduce un valor adicional denominado **salt**.

El salt consiste en una secuencia aleatoria generada para cada contraseña antes de calcular su hash. Aunque dos usuarios seleccionen exactamente la misma contraseña, la incorporación de diferentes salts produce resúmenes completamente distintos.

El salt no necesita permanecer en secreto. Puede almacenarse junto con el valor hash sin comprometer la seguridad del sistema. Su función consiste exclusivamente en garantizar que cada contraseña genere un resultado único.

Gracias al uso de salts, los ataques basados en tablas precalculadas dejan de ser prácticos, ya que el atacante tendría que reconstruir completamente dichas tablas para cada salt diferente.

5.14 Uso de Nonce

Otro concepto ampliamente utilizado en criptografía moderna es el **nonce**, abreviatura de *Number Used Once*.

Como su nombre indica, un nonce corresponde a un valor que únicamente debe utilizarse una sola vez dentro de un determinado contexto criptográfico.

Los nonces aparecen frecuentemente en protocolos de autenticación, modos de operación de cifrado y mecanismos de intercambio de llaves.

Su finalidad principal consiste en impedir ataques por repetición (*Replay Attacks*). Sin un nonce, un atacante podría capturar un mensaje válido y retransmitirlo posteriormente con el propósito de obtener acceso no autorizado o repetir una determinada operación.

Al incorporar un valor diferente en cada comunicación, el receptor puede identificar inmediatamente si un mensaje corresponde a una sesión anterior o si está siendo reutilizado de manera fraudulenta.

Es importante distinguir claramente entre un nonce y una llave criptográfica. Mientras la llave debe permanecer secreta, el nonce únicamente necesita ser único para garantizar la seguridad del protocolo.

5.15 Integración en protocolos modernos

Las funciones hash, HMAC y las funciones de derivación de llaves rara vez trabajan de manera aislada. En los sistemas actuales forman parte de arquitecturas criptográficas mucho más amplias donde interactúan con algoritmos de cifrado simétrico y mecanismos de criptografía de llave pública.

Durante el establecimiento de una conexión segura, por ejemplo, un protocolo puede utilizar criptografía asimétrica para acordar un secreto compartido entre dos participantes. Posteriormente, dicho secreto se procesa mediante una función de derivación para obtener varias llaves independientes destinadas al cifrado, autenticación e intercambio de información.

A continuación, el contenido de la comunicación se protege utilizando un algoritmo simétrico como AES, mientras que HMAC o un modo autenticado verifican la integridad de todos los mensajes transmitidos.

Esta cooperación entre diferentes primitivas criptográficas constituye uno de los principios fundamentales del diseño de protocolos modernos.

5.16 Importancia en la seguridad informática

Las funciones hash y los mecanismos de autenticación representan componentes indispensables dentro de cualquier sistema de seguridad actual. Aunque no proporcionan confidencialidad por sí mismos, complementan a los algoritmos de cifrado garantizando que

la información permanezca íntegra y que provenga de una fuente legítima.

Sin estos mecanismos, un atacante podría modificar datos cifrados, sustituir mensajes o reutilizar comunicaciones previamente capturadas sin que los participantes pudieran detectar dichas alteraciones.

El desarrollo de funciones hash resistentes, algoritmos HMAC seguros y mecanismos adecuados de derivación de llaves ha permitido construir protocolos criptográficos capaces de proteger millones de comunicaciones realizadas diariamente a través de Internet.

Capítulo 6. Diseño Criptográfico Seguro

6.1 Introducción

La seguridad de un sistema criptográfico no depende únicamente de la fortaleza matemática de los algoritmos que emplea. A lo largo de la historia han existido numerosos casos en los que algoritmos considerados seguros fueron implementados incorrectamente o utilizados de manera inapropiada, dando lugar a vulnerabilidades que permitieron comprometer sistemas completos.

El diseño criptográfico seguro constituye el conjunto de principios, metodologías y buenas prácticas orientadas a garantizar que un sistema conserve sus propiedades de seguridad incluso frente a atacantes con amplios conocimientos sobre su funcionamiento interno. Este enfoque reconoce que la criptografía moderna no debe basarse en mantener en secreto los algoritmos utilizados, sino en construir mecanismos cuya seguridad dependa exclusivamente del secreto de las llaves criptográficas.

El desarrollo de protocolos seguros implica analizar cuidadosamente aspectos como la generación de números aleatorios, la administración de llaves, la resistencia frente a ataques conocidos y la correcta integración entre los distintos

componentes criptográficos. Un error en cualquiera de estos elementos puede comprometer completamente la seguridad del sistema, independientemente de la robustez de los algoritmos empleados.

En consecuencia, el diseño criptográfico debe entenderse como un proceso integral que combina fundamentos matemáticos, ingeniería de software y análisis de amenazas para construir soluciones confiables.

6.2 Principio de Kerckhoffs

Uno de los principios fundamentales de la criptografía moderna fue formulado por Auguste Kerckhoffs en el siglo XIX y continúa siendo una referencia indispensable para el diseño de sistemas seguros.

Este principio establece que **la seguridad de un sistema criptográfico debe depender únicamente del secreto de la llave y nunca del secreto del algoritmo.**

En otras palabras, cualquier persona puede conocer completamente el funcionamiento interno del algoritmo sin que ello comprometa la seguridad de la información protegida. Incluso si un atacante dispone de toda la documentación técnica, del código fuente y de una descripción detallada del sistema, seguirá siendo incapaz de recuperar los datos mientras desconozca la llave utilizada.

Esta filosofía representa una diferencia fundamental respecto a numerosos sistemas históricos, cuya seguridad dependía de mantener oculto el método de cifrado. La experiencia demostró que este enfoque resulta poco confiable, ya que tarde o temprano los algoritmos terminan siendo conocidos.

Actualmente, todos los estándares criptográficos importantes, como AES, RSA y SHA-2, son completamente públicos y han sido analizados durante años por investigadores de todo el mundo. Su fortaleza proviene precisamente de haber resistido este escrutinio continuo.

6.3 Seguridad basada en problemas matemáticos

Los algoritmos criptográficos modernos fundamentan su seguridad en problemas matemáticos cuya resolución resulta computacionalmente inviable utilizando la tecnología disponible.

Por ejemplo, la seguridad de RSA depende de la enorme dificultad asociada a la factorización de números enteros de gran tamaño, mientras que los sistemas basados en curvas elípticas utilizan problemas relacionados con el logaritmo discreto.

En el caso de AES, la seguridad no se apoya en un único problema matemático específico, sino en la enorme complejidad generada por múltiples rondas de sustitución, permutación y mezcla de datos, las cuales dificultan el desarrollo de ataques más eficientes que una búsqueda exhaustiva de la llave.

Esta aproximación permite construir sistemas cuya resistencia puede estimarse matemáticamente y cuya seguridad aumenta al incrementar el tamaño de las llaves criptográficas.

6.4 Generación de números aleatorios

Uno de los componentes más importantes de cualquier sistema criptográfico es la generación de números aleatorios.

Las llaves criptográficas, los vectores de inicialización, los nonces y numerosos parámetros de seguridad dependen directamente de la calidad de estos valores aleatorios. Si un atacante logra predecirlos, la seguridad del sistema puede verse comprometida incluso cuando los algoritmos utilizados sean completamente seguros.

Es importante distinguir entre números pseudoaleatorios y números criptográficamente seguros.

Los generadores pseudoaleatorios tradicionales producen secuencias que aparentan ser aleatorias, pero que pueden reproducirse si se conoce su estado interno. Este comportamiento resulta adecuado para simulaciones o videojuegos, pero no para aplicaciones criptográficas.

En contraste, los generadores criptográficamente seguros están diseñados para que la predicción de futuros valores resulte computacionalmente inviable, incluso si el atacante conoce parte de la secuencia previamente generada.

Por esta razón, las implementaciones criptográficas modernas utilizan fuentes de entropía provenientes del sistema operativo y dispositivos físicos especializados para producir valores suficientemente impredecibles.

6.5 Administración de llaves criptográficas

La protección de las llaves constituye uno de los aspectos más importantes del diseño criptográfico.

Puede afirmarse que un algoritmo extremadamente robusto pierde completamente su utilidad si las llaves son almacenadas de forma insegura o transmitidas mediante canales vulnerables.

El ciclo de vida de una llave comprende varias etapas:

- Generación.
- Distribución.
- Almacenamiento.
- Utilización.
- Rotación.
- Revocación.
- Eliminación.

Cada una de estas fases debe implementarse cuidadosamente para evitar filtraciones o accesos no autorizados.

La generación debe producir llaves impredecibles utilizando fuentes confiables de aleatoriedad. Posteriormente, dichas llaves deben distribuirse mediante protocolos seguros que impidan su interceptación.

Durante el almacenamiento, las llaves suelen protegerse mediante módulos especializados o mecanismos de cifrado adicionales.

Finalmente, cuando dejan de ser necesarias, deben eliminarse de manera segura para impedir su recuperación.

La correcta administración de llaves representa uno de los mayores desafíos dentro de cualquier infraestructura criptográfica.

6.6 Separación de funciones criptográficas

Otro principio importante del diseño seguro consiste en asignar una función específica a cada algoritmo criptográfico.

El cifrado debe utilizarse exclusivamente para proteger la confidencialidad.

Las funciones hash deben emplearse para verificar la integridad.

Las firmas digitales deben utilizarse para proporcionar autenticidad y no repudio.

Las funciones de derivación deben generar nuevas llaves.

La reutilización inapropiada de un mismo algoritmo para propósitos distintos puede introducir vulnerabilidades inesperadas. Por ello, los protocolos modernos combinan múltiples primitivas criptográficas especializadas, cada una diseñada para cumplir una función claramente definida.

6.7 Errores comunes en implementaciones criptográficas

Numerosas vulnerabilidades no provienen del algoritmo en sí, sino de errores cometidos durante su implementación.

Uno de los problemas más frecuentes consiste en reutilizar la misma llave durante períodos excesivamente largos. Aunque el algoritmo permanezca seguro, aumentar el volumen de información protegida con una misma llave incrementa las oportunidades para que un atacante realice análisis criptográficos.

Otro error común consiste en reutilizar vectores de inicialización o nonces. Como se explicó anteriormente, estos

valores deben ser únicos para garantizar la seguridad de diversos modos de operación y protocolos.

También resultan problemáticos el uso de generadores pseudoaleatorios inseguros, la elección de contraseñas débiles, el almacenamiento de llaves en texto plano y la implementación incorrecta de protocolos criptográficos.

En muchos casos, estos errores permiten comprometer completamente un sistema sin necesidad de atacar directamente los algoritmos criptográficos utilizados.

6.8 Ataques considerados durante el diseño

Los protocolos criptográficos modernos deben diseñarse considerando diversos modelos de ataque.

Entre ellos se encuentran los ataques por fuerza bruta, donde el adversario intenta probar sistemáticamente todas las llaves posibles; los ataques de texto conocido, donde el atacante dispone simultáneamente del mensaje original y de su correspondiente versión cifrada; y los ataques de texto elegido, en los cuales el adversario puede solicitar el cifrado de mensajes seleccionados específicamente para facilitar el análisis del algoritmo.

También deben contemplarse ataques por repetición, ataques de intermediario (*Man-in-the-Middle*), ataques laterales basados en el consumo de energía o el tiempo de ejecución, así como vulnerabilidades derivadas de errores humanos o configuraciones incorrectas.

El diseño criptográfico seguro busca minimizar la superficie de ataque considerando todos estos escenarios desde las etapas iniciales del desarrollo.

6.9 Criptografía como parte de un sistema de seguridad

Es importante comprender que la criptografía constituye únicamente uno de los componentes de una estrategia integral de

seguridad.

Un sistema puede emplear algoritmos criptográficos extremadamente robustos y, aun así, resultar vulnerable debido a errores en la autenticación de usuarios, configuraciones incorrectas, fallos de software o ataques dirigidos contra los dispositivos donde se ejecuta.

Por ello, la criptografía debe integrarse con mecanismos adicionales como control de acceso, monitoreo, actualización de software, administración de identidades y políticas de seguridad.

Este enfoque integral permite reducir considerablemente los riesgos asociados a la protección de la información.

Capítulo 7. Criptografía Asimétrica

7.1 Introducción

La criptografía simétrica resuelve eficientemente el problema del cifrado de grandes cantidades de información. Sin embargo, presenta una limitación fundamental: tanto el emisor como el receptor deben compartir previamente una misma llave secreta. En redes pequeñas este requisito puede cumplirse sin mayores dificultades, pero en Internet, donde millones de usuarios se comunican diariamente sin conocerse, la distribución segura de llaves representa un desafío considerable.

La criptografía asimétrica fue desarrollada precisamente para resolver este problema. A diferencia de los sistemas simétricos, utiliza un **par de llaves matemáticamente relacionadas**: una llave pública, que puede compartirse libremente, y una llave privada, que debe permanecer exclusivamente bajo el control de su propietario.

Este paradigma transformó completamente la seguridad informática al permitir que dos usuarios establezcan comunicaciones seguras sin haber intercambiado previamente información secreta. Además, hizo posible el desarrollo de mecanismos de autenticación, firmas

digitales, certificados electrónicos y protocolos de seguridad utilizados actualmente en Internet.

La seguridad de estos sistemas no depende del secreto del algoritmo, sino de la enorme dificultad computacional asociada a determinados problemas matemáticos, como la factorización de números enteros o el cálculo del logaritmo discreto.

7.2 Principios de funcionamiento

En un sistema de criptografía asimétrica cada usuario genera un par de llaves.

La primera corresponde a la **llave pública**, destinada a ser distribuida libremente entre cualquier participante. Esta llave puede almacenarse en servidores, enviarse mediante correo electrónico o incorporarse dentro de certificados digitales sin comprometer la seguridad del sistema.

La segunda corresponde a la **llave privada**, la cual debe permanecer completamente protegida. Únicamente su propietario tiene acceso a ella.

Ambas llaves mantienen una relación matemática muy estrecha. Sin embargo, conocer la llave pública no permite calcular la llave privada dentro de un tiempo razonable utilizando la tecnología disponible.

Cuando un usuario desea enviar información confidencial, utiliza la llave pública del destinatario para cifrar el mensaje. Una vez cifrado, únicamente la correspondiente llave privada puede recuperar el contenido original.

Este mecanismo elimina completamente la necesidad de compartir previamente una llave secreta.

7.3 Ventajas y limitaciones

La criptografía asimétrica introdujo ventajas que modificaron profundamente la forma en que se protegen las comunicaciones digitales.

La principal ventaja consiste en resolver el problema de distribución de llaves. Los participantes ya no necesitan intercambiar secretos antes de iniciar una comunicación.

Además, permite implementar mecanismos de autenticación y firmas digitales imposibles de obtener únicamente mediante criptografía simétrica.

Sin embargo, estos algoritmos presentan un costo computacional considerablemente mayor.

Las operaciones matemáticas realizadas por RSA o por los sistemas basados en curvas elípticas requieren mucho más tiempo que las utilizadas por AES.

Por esta razón, prácticamente todos los protocolos modernos utilizan un enfoque híbrido.

Inicialmente emplean criptografía asimétrica para establecer una llave compartida.

Posteriormente utilizan algoritmos simétricos para cifrar toda la comunicación.

Este enfoque combina las ventajas de ambos paradigmas.

7.4 Intercambio de llaves Diffie-Hellman

Uno de los avances más importantes de la criptografía moderna fue el desarrollo del protocolo **Diffie-Hellman**, propuesto en 1976.

Su importancia radica en demostrar que dos personas pueden generar un secreto compartido utilizando únicamente un canal público, incluso cuando un atacante observa absolutamente toda la comunicación.

Antes de Diffie-Hellman se asumía que era imposible establecer una llave secreta sin transmitirla directamente entre los participantes.

El protocolo cambió completamente esta idea.

Su funcionamiento se basa en propiedades matemáticas relacionadas con la exponenciación modular.

Inicialmente ambos participantes acuerdan públicamente ciertos parámetros matemáticos.

Posteriormente cada uno genera un número secreto que nunca abandona su dispositivo.

A continuación realizan operaciones matemáticas utilizando dichos valores y envían únicamente los resultados públicos.

Finalmente ambos participantes pueden calcular exactamente la misma llave compartida sin haber transmitido nunca sus secretos privados.

Un observador externo conoce todos los valores intercambiados durante el protocolo, pero continúa siendo incapaz de reconstruir el secreto compartido debido a la enorme dificultad del problema matemático subyacente.

7.5 El problema del logaritmo discreto

La seguridad de Diffie-Hellman depende principalmente del denominado **problema del logaritmo discreto**.

Mientras calcular una potencia modular resulta relativamente sencillo, realizar la operación inversa constituye un problema extremadamente complejo cuando se utilizan números suficientemente grandes.

Esta diferencia entre la facilidad de una operación y la dificultad de su inversa recibe el nombre de función unidireccional (*One-Way Function*).

Actualmente no existen algoritmos prácticos capaces de resolver eficientemente este problema para tamaños de parámetros utilizados en aplicaciones reales.

Como consecuencia, el intercambio de llaves permanece seguro siempre que se utilicen parámetros suficientemente grandes y correctamente generados.

7.6 Vulnerabilidad frente al ataque Man-in-the-Middle

Aunque Diffie-Hellman permite establecer una llave compartida de forma segura, el protocolo original presenta una limitación importante.

No proporciona autenticación.

Esto significa que un atacante podría situarse entre ambos participantes e iniciar simultáneamente dos intercambios independientes.

Cada usuario creería estar comunicándose con el destinatario legítimo cuando en realidad mantiene una llave compartida únicamente con el atacante.

Este escenario recibe el nombre de **Man-in-the-Middle (MITM)**.

Para evitar esta vulnerabilidad resulta necesario combinar Diffie-Hellman con mecanismos adicionales de autenticación, como certificados digitales o firmas electrónicas.

Precisamente esta integración constituye uno de los elementos centrales de protocolos modernos como TLS.

7.7 RSA

Uno de los algoritmos de criptografía asimétrica más importantes es **RSA**, desarrollado por Ronald Rivest, Adi Shamir y Leonard Adleman.

Durante varias décadas ha constituido el sistema de llave pública más utilizado para cifrado, intercambio de llaves y firmas digitales.

La seguridad de RSA se fundamenta en la enorme dificultad asociada a la factorización de números enteros extremadamente grandes.

El algoritmo comienza seleccionando dos números primos de gran tamaño.

Posteriormente ambos valores se multiplican para obtener un nuevo número denominado módulo.

Aunque multiplicar dos números resulta sencillo, recuperar los factores originales cuando únicamente se conoce el producto representa un problema computacional extremadamente complejo.

A partir de estos valores se generan matemáticamente la llave pública y la llave privada.

La llave pública puede distribuirse libremente mientras que la llave privada permanece protegida por su propietario.

Cuando un mensaje se cifra utilizando la llave pública únicamente la correspondiente llave privada puede descifrarlo correctamente.

7.8 Generación de llaves RSA

La generación de llaves constituye uno de los procesos más importantes dentro del algoritmo RSA.

Inicialmente se seleccionan dos números primos suficientemente grandes.

Posteriormente se calcula su producto, el cual formará parte tanto de la llave pública como de la llave privada.

A continuación se determina la función totiente correspondiente y se selecciona un exponente público que cumpla determinadas propiedades matemáticas.

Finalmente se calcula el inverso modular necesario para obtener el exponente privado.

Todo este procedimiento depende directamente de los conceptos de aritmética modular estudiados en capítulos anteriores.

La correcta selección de números primos constituye un requisito indispensable para garantizar la seguridad del algoritmo.

7.9 Operaciones de cifrado y descifrado

RSA utiliza operaciones de exponenciación modular para transformar la información.

Durante el cifrado el mensaje se eleva al exponente público dentro del módulo correspondiente.

El resultado obtenido constituye el texto cifrado.

Posteriormente el destinatario aplica una segunda exponenciación utilizando el exponente privado.

Gracias a determinadas propiedades matemáticas relacionadas con la teoría de números, el resultado final coincide exactamente con el mensaje original.

Aunque este procedimiento requiere operaciones matemáticas complejas, garantiza que únicamente quien posee la llave privada pueda recuperar la información.

7.10 Limitaciones de RSA

A pesar de su enorme importancia histórica, RSA presenta ciertas limitaciones.

Su principal inconveniente consiste en el elevado costo computacional asociado a las operaciones de exponenciación modular.

Además, conforme aumenta la capacidad de procesamiento disponible resulta necesario utilizar llaves cada vez mayores para mantener un nivel adecuado de seguridad.

Esto incrementa el tiempo requerido tanto para el cifrado como para el descifrado.

Por esta razón, numerosos protocolos modernos han comenzado a reemplazar RSA por algoritmos basados en curvas elípticas, los cuales ofrecen niveles equivalentes de seguridad utilizando llaves considerablemente más pequeñas.

7.11 Criptografía de Curvas Elípticas (ECC)

La evolución de la criptografía asimétrica ha estado impulsada por la necesidad de desarrollar algoritmos que ofrezcan mayores niveles de seguridad con un menor consumo de recursos computacionales. Aunque RSA y Diffie-Hellman demostraron ser herramientas extremadamente eficaces durante varias décadas, el incremento constante de la capacidad de procesamiento obligó a

utilizar llaves cada vez más largas para mantener su resistencia frente a ataques.

Como respuesta a esta situación surgió la **Criptografía de Curvas Elípticas (Elliptic Curve Cryptography, ECC)**. Este enfoque utiliza propiedades algebraicas de las curvas elípticas definidas sobre cuerpos finitos para construir sistemas criptográficos capaces de ofrecer un nivel de seguridad equivalente al de RSA utilizando llaves considerablemente más pequeñas.

La reducción en el tamaño de las llaves representa una ventaja significativa en dispositivos con recursos limitados, como teléfonos inteligentes, tarjetas inteligentes, sensores y equipos del Internet de las Cosas. Asimismo, disminuye el tiempo requerido para realizar operaciones criptográficas y reduce el consumo de memoria y energía.

Por estas razones, la criptografía basada en curvas elípticas se ha convertido en una de las tecnologías más importantes para la protección de las comunicaciones digitales modernas.

7.12 Fundamentos matemáticos de las curvas elípticas

A pesar de su nombre, las curvas elípticas no corresponden a figuras con forma de elipse. Matemáticamente se describen mediante ecuaciones algebraicas que generan conjuntos de puntos con propiedades particulares.

Cuando estas curvas se estudian sobre cuerpos finitos, es posible definir operaciones entre sus puntos que cumplen propiedades algebraicas similares a las utilizadas en otros sistemas criptográficos. Estas operaciones permiten construir problemas matemáticos cuya resolución resulta extremadamente difícil para un atacante.

La operación fundamental consiste en la **multiplicación escalar de un punto**. A partir de un punto conocido y de un número entero, es sencillo calcular un nuevo punto sobre la curva. Sin embargo, realizar el proceso inverso, es decir, determinar el número

entero utilizado únicamente observando los puntos inicial y final, constituye un problema computacionalmente muy complejo.

Esta característica convierte a las curvas elípticas en una base adecuada para el desarrollo de algoritmos criptográficos de llave pública.

7.13 El problema del logaritmo discreto sobre curvas elípticas

La seguridad de ECC se fundamenta en el denominado **Problema del Logaritmo Discreto sobre Curvas Elípticas (Elliptic Curve Discrete Logarithm Problem, ECDLP)**.

Este problema puede describirse de la siguiente manera: dado un punto inicial sobre una curva y otro punto obtenido mediante una multiplicación escalar, resulta extremadamente difícil determinar el número entero utilizado para producir dicho resultado.

Hasta el momento no se conocen algoritmos capaces de resolver este problema de manera eficiente cuando se utilizan parámetros adecuados. En consecuencia, la seguridad de ECC depende de la enorme complejidad computacional asociada a esta operación.

Gracias a esta propiedad, un sistema basado en curvas elípticas puede ofrecer un nivel de seguridad comparable al de RSA utilizando llaves mucho más pequeñas. Esto reduce significativamente el tiempo de procesamiento y el consumo de recursos sin sacrificar la protección de la información.

7.14 Generación de llaves en ECC

El proceso de generación de llaves en un sistema basado en curvas elípticas sigue un principio similar al utilizado por otros algoritmos de criptografía asimétrica.

Inicialmente se selecciona un número entero aleatorio que constituye la **llave privada**. Posteriormente, mediante una multiplicación escalar sobre un punto generador previamente definido, se obtiene el punto correspondiente que funcionará como **llave pública**.

La relación matemática existente entre ambas llaves permite que la pública pueda distribuirse libremente sin revelar información suficiente para reconstruir la privada.

La seguridad del sistema depende precisamente de la dificultad del problema del logaritmo discreto sobre curvas elípticas, el cual impide calcular la llave privada a partir de la llave pública utilizando métodos computacionalmente viables.

7.15 ECDH (Elliptic Curve Diffie-Hellman)

Uno de los algoritmos más importantes derivados de ECC es **Elliptic Curve Diffie-Hellman (ECDH)**.

Su objetivo es idéntico al del protocolo Diffie-Hellman tradicional: permitir que dos participantes generen un secreto compartido sin transmitirlo directamente a través del canal de comunicación.

La diferencia radica en que ECDH reemplaza las operaciones de exponenciación modular por operaciones realizadas sobre curvas elípticas.

Cada participante genera un par de llaves pública y privada. Posteriormente intercambian únicamente sus llaves públicas y realizan operaciones matemáticas utilizando su propia llave privada y la llave pública recibida.

Como resultado, ambos obtienen exactamente el mismo secreto compartido sin haber transmitido nunca dicho valor.

Este secreto suele utilizarse posteriormente como entrada para una función de derivación de llaves, generando las llaves simétricas empleadas durante la comunicación.

Actualmente, ECDH constituye uno de los mecanismos de intercambio de llaves más utilizados en protocolos modernos debido a su elevada eficiencia y a la menor longitud de las llaves requeridas.

7.16 ECDSA (Elliptic Curve Digital Signature Algorithm)

Además del intercambio de llaves, las curvas elípticas también permiten implementar mecanismos de firma digital.

El algoritmo más utilizado con este propósito es **Elliptic Curve Digital Signature Algorithm (ECDSA)**.

Su funcionamiento se basa en el uso de una llave privada para generar una firma matemática asociada al contenido de un mensaje. Posteriormente, cualquier participante que conozca la correspondiente llave pública puede verificar la autenticidad de dicha firma sin necesidad de acceder a la llave privada.

La seguridad de ECDSA depende igualmente del problema del logaritmo discreto sobre curvas elípticas.

Gracias a la eficiencia de estas operaciones, ECDSA permite generar firmas considerablemente más pequeñas que las obtenidas mediante RSA, manteniendo un nivel equivalente de seguridad.

Por esta razón, numerosos certificados digitales y protocolos modernos emplean actualmente firmas basadas en curvas elípticas.

7.17 Comparación entre RSA y ECC

Aunque RSA y ECC pertenecen a la misma categoría de algoritmos de criptografía asimétrica, presentan diferencias importantes.

RSA fundamenta su seguridad en la dificultad de factorizar números enteros extremadamente grandes. En cambio, ECC depende del problema del logaritmo discreto sobre curvas elípticas.

Como consecuencia, ECC puede alcanzar niveles similares de seguridad utilizando llaves mucho más pequeñas. Esto reduce el tiempo requerido para la generación de llaves, el establecimiento de conexiones y la creación de firmas digitales.

Asimismo, el menor tamaño de las llaves disminuye el consumo de memoria y el ancho de banda necesario para transmitir información

criptográfica, aspectos especialmente importantes en dispositivos móviles y sistemas embebidos.

No obstante, RSA continúa siendo ampliamente utilizado debido a su extensa trayectoria, su gran compatibilidad con sistemas existentes y la amplia experiencia acumulada durante décadas de implementación.

Actualmente, numerosos protocolos admiten ambos algoritmos, permitiendo seleccionar el más adecuado según las características de la aplicación.

7.18 Aplicaciones actuales de ECC

La criptografía basada en curvas elípticas se encuentra presente en una gran variedad de aplicaciones.

Entre las más importantes destacan los protocolos de comunicación segura utilizados en Internet, donde ECDH permite establecer llaves compartidas y ECDSA proporciona mecanismos de autenticación mediante firmas digitales.

Asimismo, ECC se emplea ampliamente en dispositivos móviles, tarjetas inteligentes, sistemas bancarios, criptomonedas, infraestructura de clave pública y numerosos servicios de autenticación.

Su elevada eficiencia ha favorecido su adopción en dispositivos con recursos limitados, donde la reducción del consumo energético y del tiempo de procesamiento representa una ventaja significativa.

7.19 Comparación general de los algoritmos asimétricos

Los algoritmos estudiados en este capítulo resuelven diferentes necesidades dentro de la criptografía moderna.

Diffie-Hellman y **ECDH** están orientados principalmente al establecimiento seguro de secretos compartidos entre dos participantes.

RSA permite tanto el cifrado como la generación de firmas digitales, además de utilizarse históricamente para el intercambio de llaves.

ECDSA se especializa en la generación de firmas digitales utilizando curvas elípticas.

En la práctica, estos algoritmos rara vez trabajan de forma aislada. Los protocolos modernos combinan varias de estas técnicas para construir sistemas completos de autenticación, intercambio de llaves y protección de la información.

Capítulo 8. Firmas Digitales, Certificados Digitales y TLS 1.3

8.1 Introducción

La criptografía moderna no solo busca proteger la confidencialidad de la información. En la práctica, los sistemas informáticos requieren garantizar que los mensajes realmente provienen del remitente esperado, que no han sido modificados durante la transmisión y que el autor de una transacción no pueda negar posteriormente haberla realizado. Estas propiedades se conocen como autenticidad, integridad y no repudio, respectivamente.

Los algoritmos estudiados en los capítulos anteriores permiten construir mecanismos capaces de satisfacer estos requisitos. Las funciones hash proporcionan integridad; la criptografía asimétrica hace posible la autenticación; y la combinación de ambas conduce al desarrollo de las firmas digitales y de la infraestructura de certificados utilizada actualmente en Internet.

Además, los protocolos modernos integran todos estos mecanismos dentro de arquitecturas complejas que permiten establecer conexiones seguras entre millones de dispositivos alrededor del mundo. Uno de los ejemplos más importantes es **Transport Layer**

Security (TLS) 1.3, protocolo responsable de proteger la mayor parte del tráfico web actual.

8.2 Firmas digitales

Una **firma digital** es un mecanismo criptográfico que permite demostrar la autenticidad de un mensaje o documento electrónico mediante el uso de criptografía asimétrica.

Su funcionamiento es conceptualmente similar al de una firma manuscrita. Sin embargo, mientras una firma tradicional puede falsificarse mediante técnicas relativamente sencillas, una firma digital depende de operaciones matemáticas cuya falsificación resulta computacionalmente inviable cuando se utilizan algoritmos seguros.

El proceso comienza calculando el resumen criptográfico del documento mediante una función hash. Posteriormente, dicho resumen se procesa utilizando la llave privada del firmante. El resultado constituye la firma digital.

Cuando otro usuario recibe el documento, calcula nuevamente el hash del contenido y verifica la firma utilizando la llave pública correspondiente. Si ambos resultados coinciden, puede concluir que el documento no fue modificado y que únicamente el propietario de la llave privada pudo haber generado dicha firma.

Este procedimiento evita cifrar completamente el documento con la llave privada, lo que reduciría considerablemente la eficiencia del sistema. En su lugar, únicamente se firma el resumen criptográfico, manteniendo un elevado nivel de seguridad con un menor costo computacional.

8.3 Propiedades de las firmas digitales

Las firmas digitales proporcionan tres propiedades fundamentales.

La primera es la **autenticidad**, ya que permiten verificar la identidad del autor del documento. Si la firma se valida correctamente utilizando la llave pública correspondiente, el

receptor puede tener confianza en que el mensaje fue generado por quien afirma haberlo enviado.

La segunda propiedad corresponde a la **integridad**. Debido a que la firma se calcula sobre el resumen criptográfico del documento, cualquier modificación realizada posteriormente altera el valor hash y provoca que la verificación falle inmediatamente.

Finalmente, las firmas digitales proporcionan **no repudio**. Dado que únicamente el propietario posee la llave privada utilizada durante la firma, posteriormente no puede negar razonablemente haber firmado el documento, siempre que dicha llave haya permanecido protegida.

Estas propiedades convierten a las firmas digitales en uno de los mecanismos más importantes dentro de la seguridad informática moderna.

8.4 Aplicaciones de las firmas digitales

Las firmas digitales poseen numerosas aplicaciones en la actualidad.

Se utilizan para firmar documentos electrónicos, contratos digitales, certificados académicos, comprobantes fiscales, transacciones bancarias y software distribuido por fabricantes.

También desempeñan un papel fundamental en la distribución de actualizaciones de sistemas operativos. Antes de instalar una actualización, el dispositivo verifica la firma digital del fabricante para confirmar que el software realmente proviene de una fuente confiable y que no fue modificado durante su distribución.

Asimismo, numerosas plataformas gubernamentales utilizan firmas digitales para autenticar trámites electrónicos y garantizar la validez jurídica de documentos transmitidos mediante Internet.

8.5 Certificados digitales

Aunque las firmas digitales permiten verificar la autenticidad de un documento, aún existe una pregunta importante: **¿cómo puede un usuario estar seguro de que una determinada llave pública realmente pertenece a la persona o al servidor que afirma poseerla?**

La respuesta se encuentra en los **certificados digitales**.

Un certificado digital es un documento electrónico que vincula una identidad con una llave pública. Dicho documento contiene información como el nombre del propietario, la llave pública correspondiente, el período de validez y diversos datos administrativos.

Lo más importante es que el certificado se encuentra firmado digitalmente por una entidad de confianza denominada **Autoridad Certificadora (Certificate Authority o CA)**.

Gracias a esta firma, cualquier usuario puede verificar que la llave pública incluida dentro del certificado fue validada previamente por una entidad reconocida.

8.6 Autoridades certificadoras

Las **Autoridades Certificadoras (CA)** constituyen organizaciones responsables de emitir y firmar certificados digitales.

Antes de emitir un certificado, la autoridad verifica la identidad del solicitante siguiendo procedimientos previamente establecidos. Una vez completada dicha verificación, genera un certificado que contiene la llave pública del propietario y lo firma utilizando su propia llave privada.

Los sistemas operativos y los navegadores incorporan una lista de autoridades certificadoras consideradas confiables.

Cuando un usuario visita un sitio web seguro, su navegador verifica automáticamente la firma del certificado recibido. Si la firma pertenece a una autoridad reconocida y el certificado permanece vigente, la conexión puede establecerse normalmente.

En caso contrario, el navegador advierte al usuario sobre la posible existencia de riesgos de seguridad.

8.7 Cadena de confianza

La confianza en los certificados digitales se basa en un modelo jerárquico conocido como **cadena de confianza**.

En la parte superior de esta estructura se encuentran las **Autoridades Raíz**, cuyos certificados son instalados previamente dentro del sistema operativo o del navegador.

A partir de ellas se emiten certificados intermedios que, a su vez, firman certificados destinados a servidores, organizaciones o usuarios finales.

Durante el establecimiento de una conexión segura, el navegador verifica sucesivamente cada uno de los certificados presentes en la cadena hasta llegar finalmente a una autoridad raíz considerada confiable.

Si cualquiera de las firmas resulta inválida o alguno de los certificados ha expirado o sido revocado, la cadena completa deja de ser confiable y la conexión es rechazada.

8.8 TLS 1.3

Uno de los protocolos más importantes desarrollados para proteger las comunicaciones en Internet es **Transport Layer Security (TLS)**.

TLS proporciona confidencialidad, integridad y autenticación durante la transmisión de información entre dos participantes.

Su versión más reciente, **TLS 1.3**, incorpora importantes mejoras tanto en seguridad como en rendimiento respecto a versiones anteriores.

Cuando un usuario accede a un sitio web mediante HTTPS, el navegador y el servidor ejecutan inicialmente un proceso denominado **Handshake**, cuyo objetivo consiste en establecer una comunicación segura antes de intercambiar cualquier dato sensible.

Durante este procedimiento ambas partes acuerdan los algoritmos criptográficos que utilizarán, verifican la identidad del servidor mediante certificados digitales y generan un secreto compartido utilizando mecanismos de criptografía asimétrica.

Posteriormente dicho secreto se transforma mediante funciones de derivación en las llaves simétricas utilizadas para proteger toda la comunicación.

8.9 Funcionamiento del Handshake

El establecimiento de una conexión TLS puede dividirse en varias etapas.

Inicialmente el cliente envía un mensaje indicando las versiones del protocolo y los algoritmos criptográficos que soporta.

El servidor responde seleccionando la configuración más adecuada y envía además su certificado digital.

El cliente verifica la autenticidad del certificado comprobando la cadena de confianza correspondiente.

Posteriormente ambas partes ejecutan un algoritmo de intercambio de llaves, generalmente **ECDH**, obteniendo un secreto compartido sin transmitirlo directamente por la red.

Este secreto se procesa mediante una función de derivación de llaves para generar las llaves simétricas que protegerán la comunicación.

Finalmente ambos participantes intercambian mensajes autenticados confirmando que el establecimiento de la sesión fue exitoso.

A partir de ese momento toda la información se transmite utilizando algoritmos simétricos como AES-GCM.

8.10 Algoritmos utilizados por TLS 1.3

TLS 1.3 combina diversas primitivas criptográficas estudiadas a lo largo de este trabajo.

Las funciones hash permiten verificar la integridad de los mensajes intercambiados durante el protocolo.

Los algoritmos ECDH realizan el intercambio seguro del secreto compartido.

Las funciones de derivación generan múltiples llaves independientes destinadas al cifrado y autenticación.

Finalmente, algoritmos simétricos como **AES-GCM** protegen toda la información transmitida entre el cliente y el servidor.

Esta integración demuestra que la seguridad moderna no depende de un único algoritmo, sino de la cooperación coordinada entre múltiples mecanismos criptográficos.

8.11 Ventajas de TLS 1.3

TLS 1.3 introduce mejoras importantes respecto a versiones anteriores del protocolo.

Una de las más relevantes consiste en la eliminación de algoritmos considerados inseguros o vulnerables.

Asimismo, reduce el número de mensajes necesarios durante el establecimiento de la conexión, disminuyendo la latencia y mejorando el rendimiento general.

El protocolo también favorece el uso de algoritmos modernos como ECDH y AES-GCM, proporcionando un mayor nivel de seguridad frente a ataques conocidos.

Estas mejoras han convertido a TLS 1.3 en el estándar predominante para la protección de comunicaciones web.

Conclusiones Generales

El estudio de la criptografía moderna demuestra que la protección de la información no depende de un único algoritmo, sino de la integración de múltiples mecanismos matemáticos diseñados para resolver problemas específicos de seguridad. Desde los primeros sistemas clásicos hasta protocolos como TLS 1.3, la evolución de

la disciplina ha estado marcada por la necesidad de responder a nuevos desafíos tecnológicos y a métodos de ataque cada vez más sofisticados.

Los fundamentos matemáticos, como la aritmética modular y la teoría de números, constituyen la base sobre la cual se construyen algoritmos de cifrado, funciones hash y sistemas de llave pública. Sobre estos principios se desarrollan mecanismos capaces de garantizar confidencialidad, integridad, autenticidad y no repudio, propiedades indispensables para la seguridad de las comunicaciones digitales.

Asimismo, el análisis de algoritmos como AES, Diffie-Hellman, RSA y la criptografía de curvas elípticas evidencia que cada uno cumple una función específica dentro de los sistemas modernos. La combinación adecuada de estas herramientas permite construir protocolos robustos que protegen millones de transacciones diarias en Internet.

Finalmente, el estudio de TLS 1.3 muestra cómo las distintas primitivas criptográficas trabajan de forma coordinada para establecer conexiones seguras entre dispositivos que nunca antes han compartido información secreta. Este protocolo representa un ejemplo claro de la evolución de la criptografía desde técnicas de cifrado aisladas hasta infraestructuras complejas orientadas a garantizar la seguridad integral de la información en un entorno global.

Referencias

López G., M. (2026). *Introducción a la criptografía* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Representación de la información* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Aritmética modular* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Inversos modulares* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Criptografía clásica* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *One-Time Pad* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Advanced Encryption Standard (AES)* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *AES computacional* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Modos de operación de cifrados por bloque* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Funciones hash* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Message Authentication Code (MAC)* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Hash-based Message Authentication Code (HMAC)* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Diseño criptográfico seguro* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Diffie-Hellman* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *RSA* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Criptografía de curvas elípticas* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *Firmas digitales* [Presentación de PowerPoint]. Universidad Panamericana.

López G., M. (2026). *TLS 1.3* [Presentación de PowerPoint]. Universidad Panamericana.